

BreastCancer(1) (3)

October 7, 2026

```
[1]: import pandas as pd
      from sklearn.datasets import load_breast_cancer
      import seaborn as sns
```

```
[2]: df = load_breast_cancer(as_frame = True).frame
      df
```

```
[2]:
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	
..	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	
565	20.13	28.25	131.20	1261.0	0.09780	
566	16.60	28.08	108.30	858.1	0.08455	
567	20.60	29.33	140.10	1265.0	0.11780	
568	7.76	24.54	47.92	181.0	0.05263	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.30010	0.14710	0.2419	
1	0.07864	0.08690	0.07017	0.1812	
2	0.15990	0.19740	0.12790	0.2069	
3	0.28390	0.24140	0.10520	0.2597	
4	0.13280	0.19800	0.10430	0.1809	
..	...	...	...	...	
564	0.11590	0.24390	0.13890	0.1726	
565	0.10340	0.14400	0.09791	0.1752	
566	0.10230	0.09251	0.05302	0.1590	
567	0.27700	0.35140	0.15200	0.2397	
568	0.04362	0.00000	0.00000	0.1587	

	mean fractal dimension	...	worst texture	worst perimeter	worst area	\
0	0.07871	...	17.33	184.60	2019.0	
1	0.05667	...	23.41	158.80	1956.0	
2	0.05999	...	25.53	152.50	1709.0	

3	0.09744	...	26.50	98.87	567.7
4	0.05883	...	16.67	152.20	1575.0
..	...	...	...	...	...
564	0.05623	...	26.40	166.10	2027.0
565	0.05533	...	38.25	155.00	1731.0
566	0.05648	...	34.12	126.70	1124.0
567	0.07016	...	39.42	184.60	1821.0
568	0.05884	...	30.37	59.16	268.6

	worst smoothness	worst compactness	worst concavity \
0	0.16220	0.66560	0.7119
1	0.12380	0.18660	0.2416
2	0.14440	0.42450	0.4504
3	0.20980	0.86630	0.6869
4	0.13740	0.20500	0.4000
..	...	...	...
564	0.14100	0.21130	0.4107
565	0.11660	0.19220	0.3215
566	0.11390	0.30940	0.3403
567	0.16500	0.86810	0.9387
568	0.08996	0.06444	0.0000

	worst concave points	worst symmetry	worst fractal dimension	target
0	0.2654	0.4601	0.11890	0
1	0.1860	0.2750	0.08902	0
2	0.2430	0.3613	0.08758	0
3	0.2575	0.6638	0.17300	0
4	0.1625	0.2364	0.07678	0
..	...	...	...	...
564	0.2216	0.2060	0.07115	0
565	0.1628	0.2572	0.06637	0
566	0.1418	0.2218	0.07820	0
567	0.2650	0.4087	0.12400	0
568	0.0000	0.2871	0.07039	1

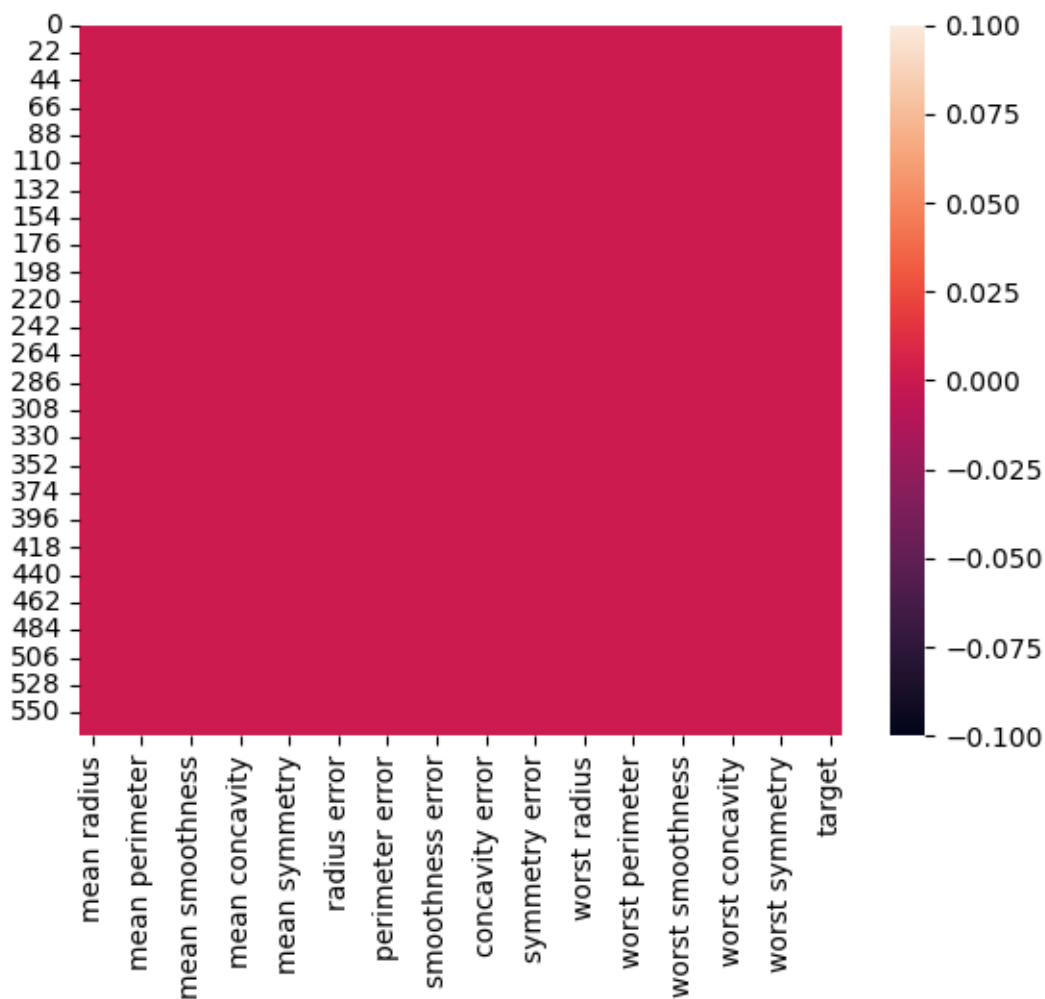
[569 rows x 31 columns]

```
[3]: df.shape # shows the shape of the dataframe
```

```
[3]: (569, 31)
```

```
[4]: sns.heatmap(df.isnull())
```

```
[4]: <Axes: >
```



```
[5]: df.isnull().sum() # no null records found in the columns shown
```

```
[5]: mean radius          0
     mean texture         0
     mean perimeter       0
     mean area            0
     mean smoothness      0
     mean compactness     0
     mean concavity       0
     mean concave points  0
     mean symmetry        0
     mean fractal dimension 0
     radius error         0
     texture error        0
     perimeter error      0
```

```

area error          0
smoothness error    0
compactness error    0
concavity error      0
concave points error 0
symmetry error       0
fractal dimension error 0
worst radius         0
worst texture        0
worst perimeter      0
worst area           0
worst smoothness     0
worst compactness    0
worst concavity      0
worst concave points 0
worst symmetry        0
worst fractal dimension 0
target              0
dtype: int64

```

```
[6]: df.info() # shows the general information about the dataset
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   mean radius                          569 non-null    float64
1   mean texture                         569 non-null    float64
2   mean perimeter                      569 non-null    float64
3   mean area                          569 non-null    float64
4   mean smoothness                     569 non-null    float64
5   mean compactness                    569 non-null    float64
6   mean concavity                      569 non-null    float64
7   mean concave points                 569 non-null    float64
8   mean symmetry                      569 non-null    float64
9   mean fractal dimension              569 non-null    float64
10  radius error                        569 non-null    float64
11  texture error                      569 non-null    float64
12  perimeter error                    569 non-null    float64
13  area error                         569 non-null    float64
14  smoothness error                   569 non-null    float64
15  compactness error                  569 non-null    float64
16  concavity error                    569 non-null    float64
17  concave points error               569 non-null    float64
18  symmetry error                     569 non-null    float64
19  fractal dimension error            569 non-null    float64
20  worst radius                       569 non-null    float64

```

```

21 worst texture          569 non-null    float64
22 worst perimeter        569 non-null    float64
23 worst area             569 non-null    float64
24 worst smoothness       569 non-null    float64
25 worst compactness      569 non-null    float64
26 worst concavity        569 non-null    float64
27 worst concave points   569 non-null    float64
28 worst symmetry         569 non-null    float64
29 worst fractal dimension 569 non-null    float64
30 target                 569 non-null    int64

```

dtypes: float64(30), int64(1)

memory usage: 137.9 KB

```
[7]: df.drop_duplicates() # dropping the duplicates in the dataframe
```

```
[7]:
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	
..	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	
565	20.13	28.25	131.20	1261.0	0.09780	
566	16.60	28.08	108.30	858.1	0.08455	
567	20.60	29.33	140.10	1265.0	0.11780	
568	7.76	24.54	47.92	181.0	0.05263	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.30010	0.14710	0.2419	
1	0.07864	0.08690	0.07017	0.1812	
2	0.15990	0.19740	0.12790	0.2069	
3	0.28390	0.24140	0.10520	0.2597	
4	0.13280	0.19800	0.10430	0.1809	
..	...	...	...	...	
564	0.11590	0.24390	0.13890	0.1726	
565	0.10340	0.14400	0.09791	0.1752	
566	0.10230	0.09251	0.05302	0.1590	
567	0.27700	0.35140	0.15200	0.2397	
568	0.04362	0.00000	0.00000	0.1587	

	mean fractal dimension	...	worst texture	worst perimeter	worst area	\
0	0.07871	...	17.33	184.60	2019.0	
1	0.05667	...	23.41	158.80	1956.0	
2	0.05999	...	25.53	152.50	1709.0	
3	0.09744	...	26.50	98.87	567.7	
4	0.05883	...	16.67	152.20	1575.0	

..	...	...	...	...	...
564	0.05623	...	26.40	166.10	2027.0
565	0.05533	...	38.25	155.00	1731.0
566	0.05648	...	34.12	126.70	1124.0
567	0.07016	...	39.42	184.60	1821.0
568	0.05884	...	30.37	59.16	268.6

	worst smoothness	worst compactness	worst concavity \
0	0.16220	0.66560	0.7119
1	0.12380	0.18660	0.2416
2	0.14440	0.42450	0.4504
3	0.20980	0.86630	0.6869
4	0.13740	0.20500	0.4000
..	...	...	...
564	0.14100	0.21130	0.4107
565	0.11660	0.19220	0.3215
566	0.11390	0.30940	0.3403
567	0.16500	0.86810	0.9387
568	0.08996	0.06444	0.0000

	worst concave points	worst symmetry	worst fractal dimension	target
0	0.2654	0.4601	0.11890	0
1	0.1860	0.2750	0.08902	0
2	0.2430	0.3613	0.08758	0
3	0.2575	0.6638	0.17300	0
4	0.1625	0.2364	0.07678	0
..	...	...	...	...
564	0.2216	0.2060	0.07115	0
565	0.1628	0.2572	0.06637	0
566	0.1418	0.2218	0.07820	0
567	0.2650	0.4087	0.12400	0
568	0.0000	0.2871	0.07039	1

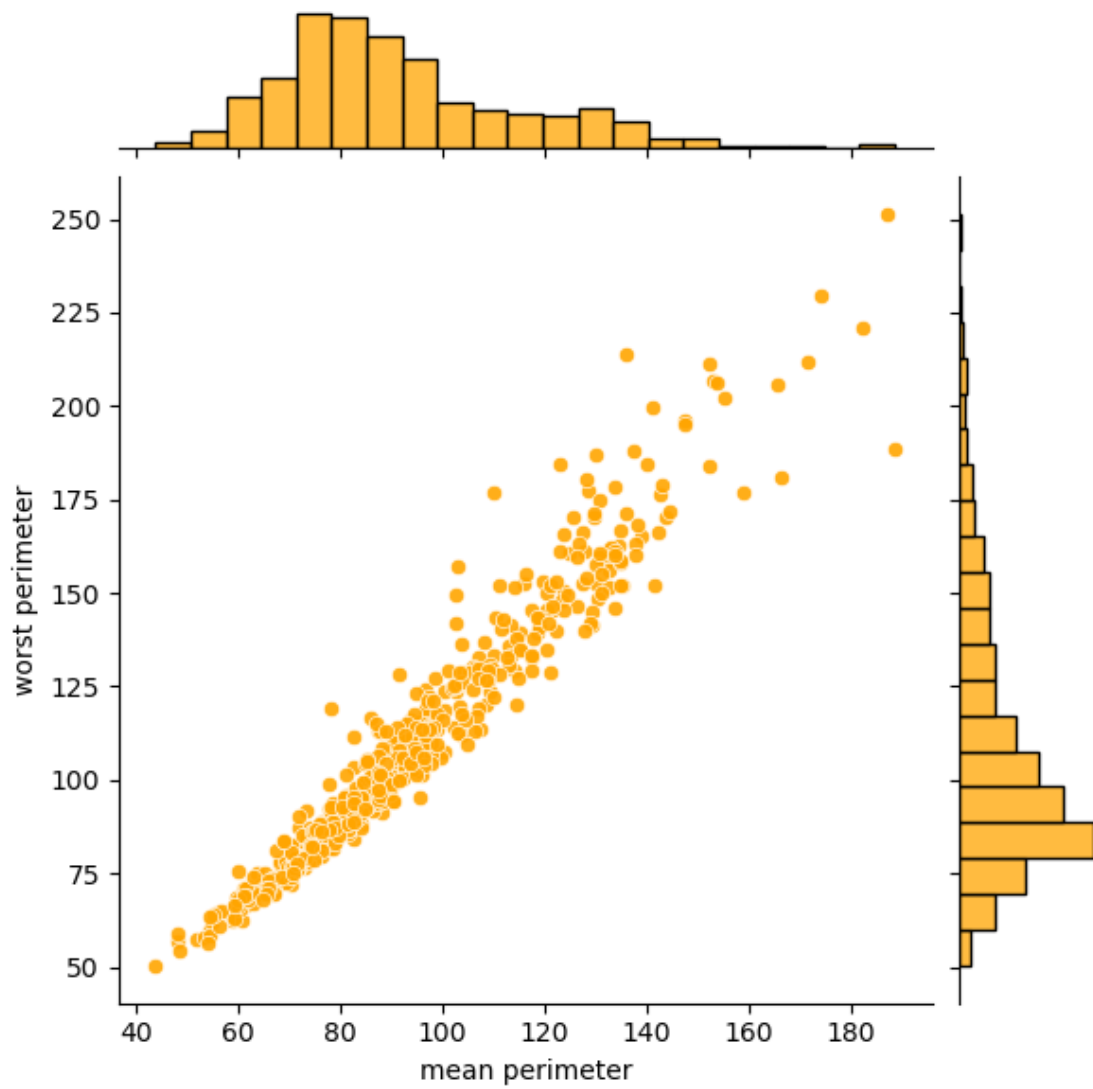
[569 rows x 31 columns]

```
[8]: df["target"].nunique() # shows the total unique values in the target column
```

```
[8]: 2
```

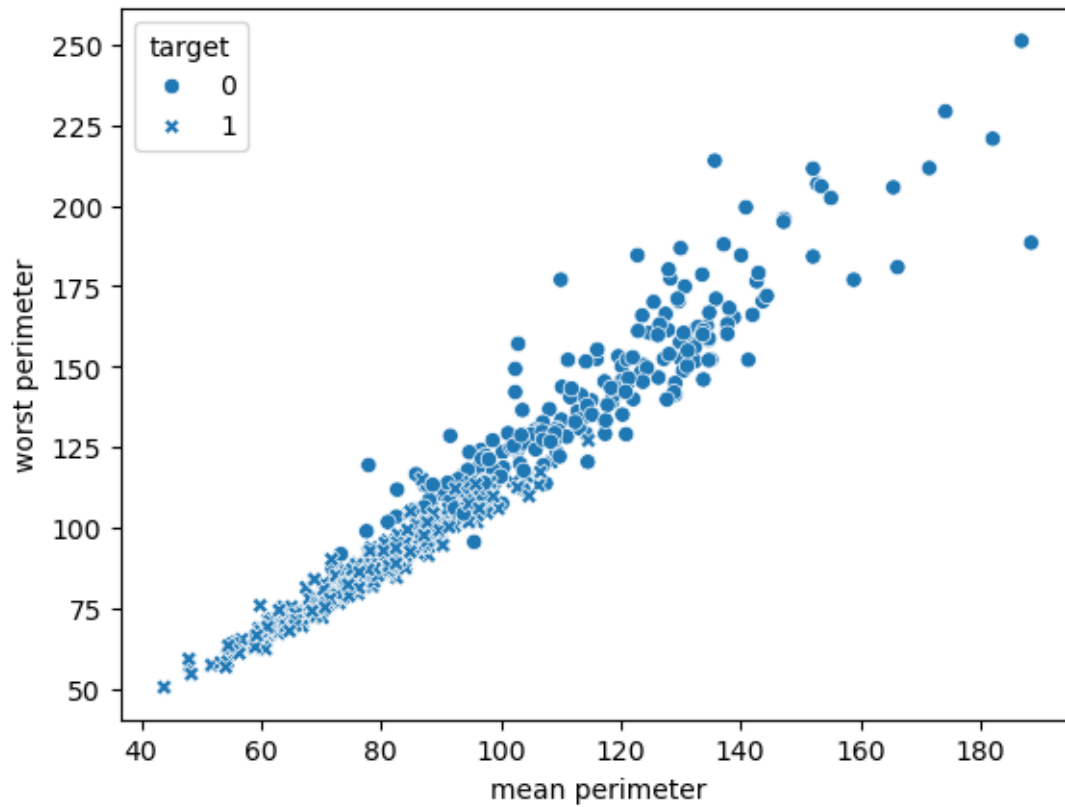
```
[9]: # EDA Explanatory Data Analysis using jointplot
sns.jointplot(x = "mean perimeter",
              y = "worst perimeter", data = df, alpha = 0.9, color = 'orange')
```

```
[9]: <seaborn.axisgrid.JointGrid at 0x2051c263230>
```



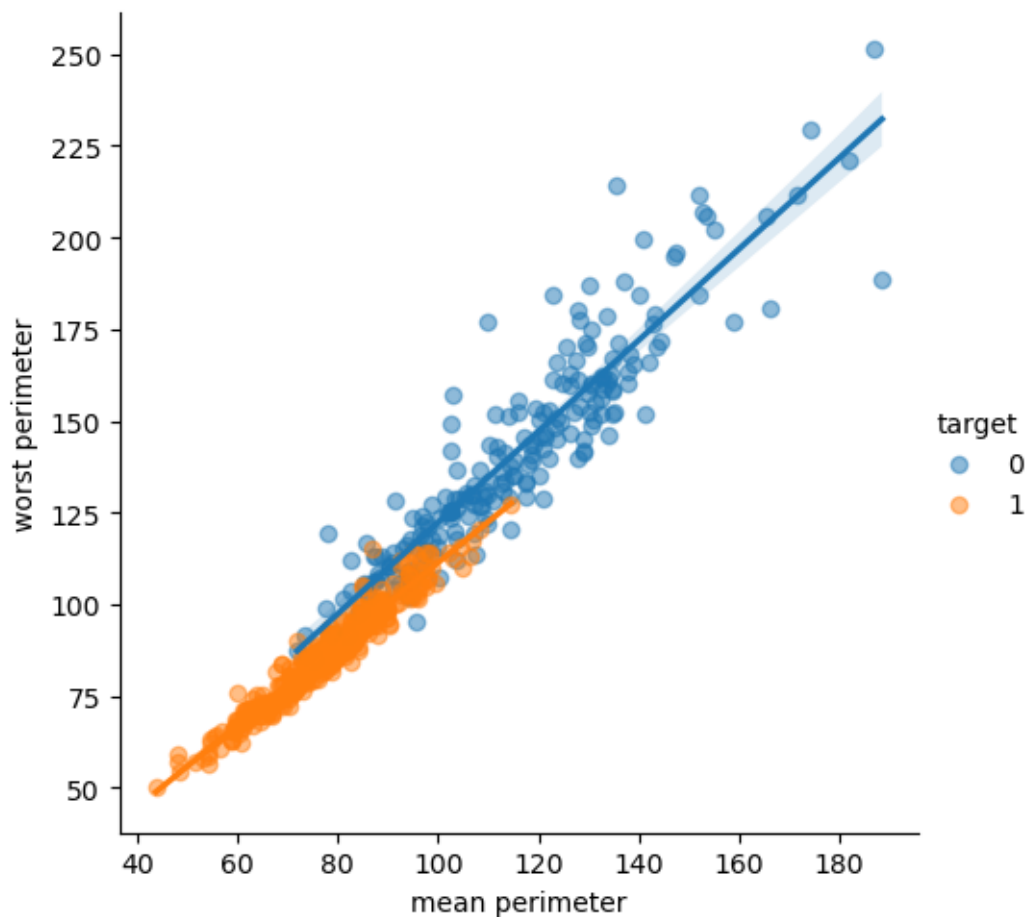
```
[10]: sns.scatterplot( data = df, x ="mean perimeter", y = "worst perimeter", style =_
↪ "target" )
```

```
[10]: <Axes: xlabel='mean perimeter', ylabel='worst perimeter'>
```



```
[11]: sns.lmplot (x = "mean perimeter", # show the X value  
                y = "worst perimeter", # shows the y value  
                scatter_kws = {"alpha" : 0.5 }, # plots the scatter plot  
                data = df, hue = "target")
```

```
[11]: <seaborn.axisgrid.FacetGrid at 0x2051d4b8c20>
```



```
[12]: df.describe()
```

```
[12]:
```

	mean radius	mean texture	mean perimeter	mean area \
count	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104
std	3.524049	4.301036	24.298981	351.914129
min	6.981000	9.710000	43.790000	143.500000
25%	11.700000	16.170000	75.170000	420.300000
50%	13.370000	18.840000	86.240000	551.100000
75%	15.780000	21.800000	104.100000	782.700000
max	28.110000	39.280000	188.500000	2501.000000

	mean smoothness	mean compactness	mean concavity	mean concave points \
count	569.000000	569.000000	569.000000	569.000000
mean	0.096360	0.104341	0.088799	0.048919
std	0.014064	0.052813	0.079720	0.038803
min	0.052630	0.019380	0.000000	0.000000
25%	0.086370	0.064920	0.029560	0.020310

50%	0.095870	0.092630	0.061540	0.033500
75%	0.105300	0.130400	0.130700	0.074000
max	0.163400	0.345400	0.426800	0.201200

	mean symmetry	mean fractal dimension	...	worst texture	\
count	569.000000	569.000000	...	569.000000	
mean	0.181162	0.062798	...	25.677223	
std	0.027414	0.007060	...	6.146258	
min	0.106000	0.049960	...	12.020000	
25%	0.161900	0.057700	...	21.080000	
50%	0.179200	0.061540	...	25.410000	
75%	0.195700	0.066120	...	29.720000	
max	0.304000	0.097440	...	49.540000	

	worst perimeter	worst area	worst smoothness	worst compactness	\
count	569.000000	569.000000	569.000000	569.000000	
mean	107.261213	880.583128	0.132369	0.254265	
std	33.602542	569.356993	0.022832	0.157336	
min	50.410000	185.200000	0.071170	0.027290	
25%	84.110000	515.300000	0.116600	0.147200	
50%	97.660000	686.500000	0.131300	0.211900	
75%	125.400000	1084.000000	0.146000	0.339100	
max	251.200000	4254.000000	0.222600	1.058000	

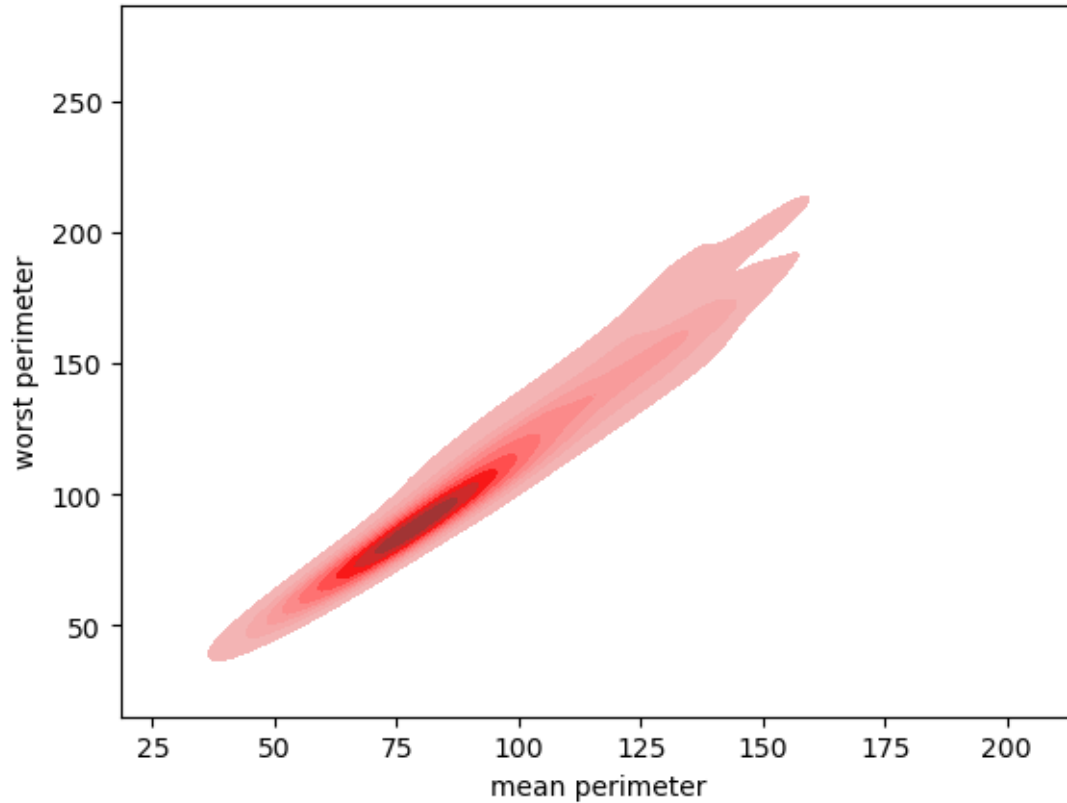
	worst concavity	worst concave points	worst symmetry	\
count	569.000000	569.000000	569.000000	
mean	0.272188	0.114606	0.290076	
std	0.208624	0.065732	0.061867	
min	0.000000	0.000000	0.156500	
25%	0.114500	0.064930	0.250400	
50%	0.226700	0.099930	0.282200	
75%	0.382900	0.161400	0.317900	
max	1.252000	0.291000	0.663800	

	worst fractal dimension	target
count	569.000000	569.000000
mean	0.083946	0.627417
std	0.018061	0.483918
min	0.055040	0.000000
25%	0.071460	0.000000
50%	0.080040	1.000000
75%	0.092080	1.000000
max	0.207500	1.000000

[8 rows x 31 columns]

```
[13]: sns.kdeplot(x = "mean perimeter",y = "worst perimeter", fill = True ,data = df ,
↳ color = 'red')
```

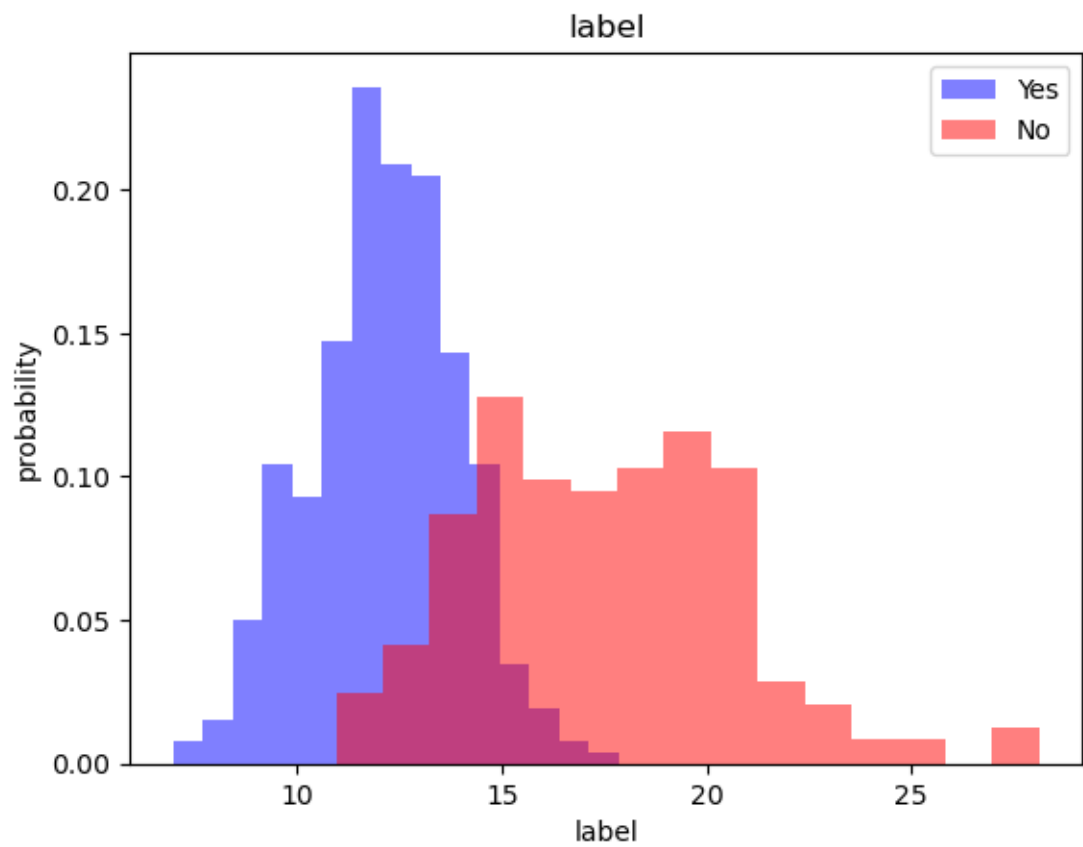
```
[13]: <Axes: xlabel='mean perimeter', ylabel='worst perimeter'>
```

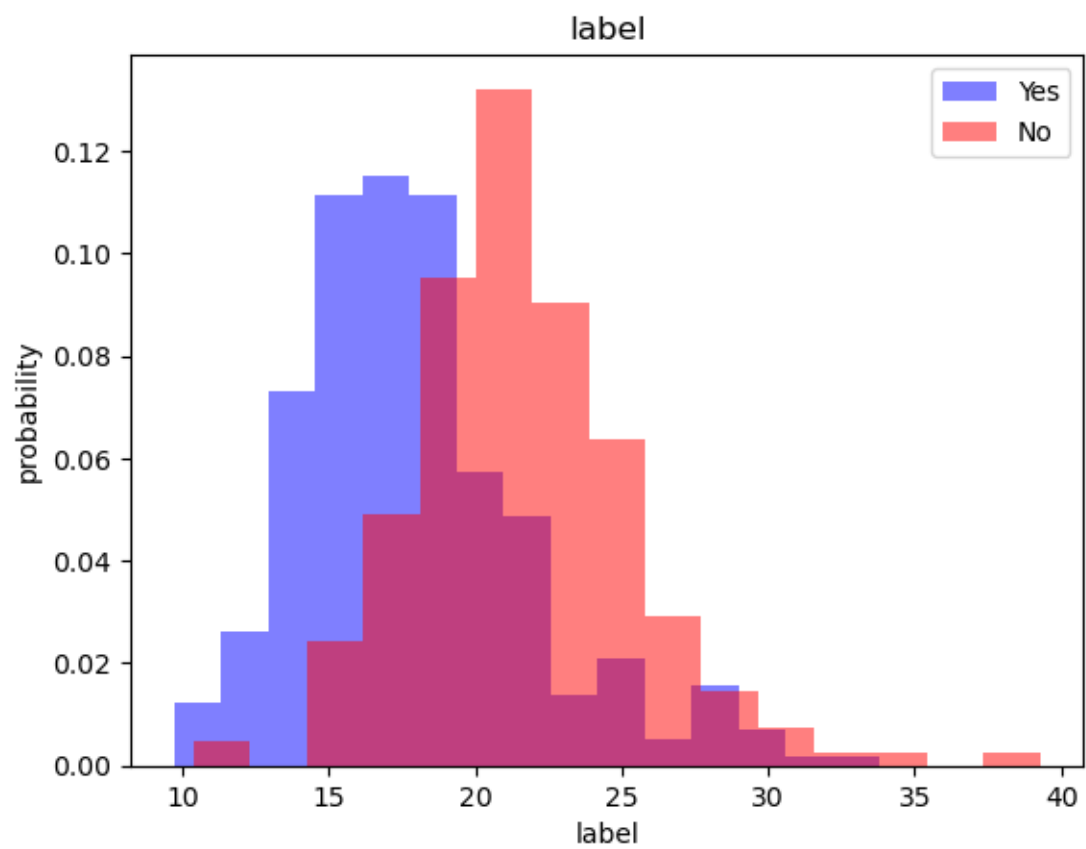


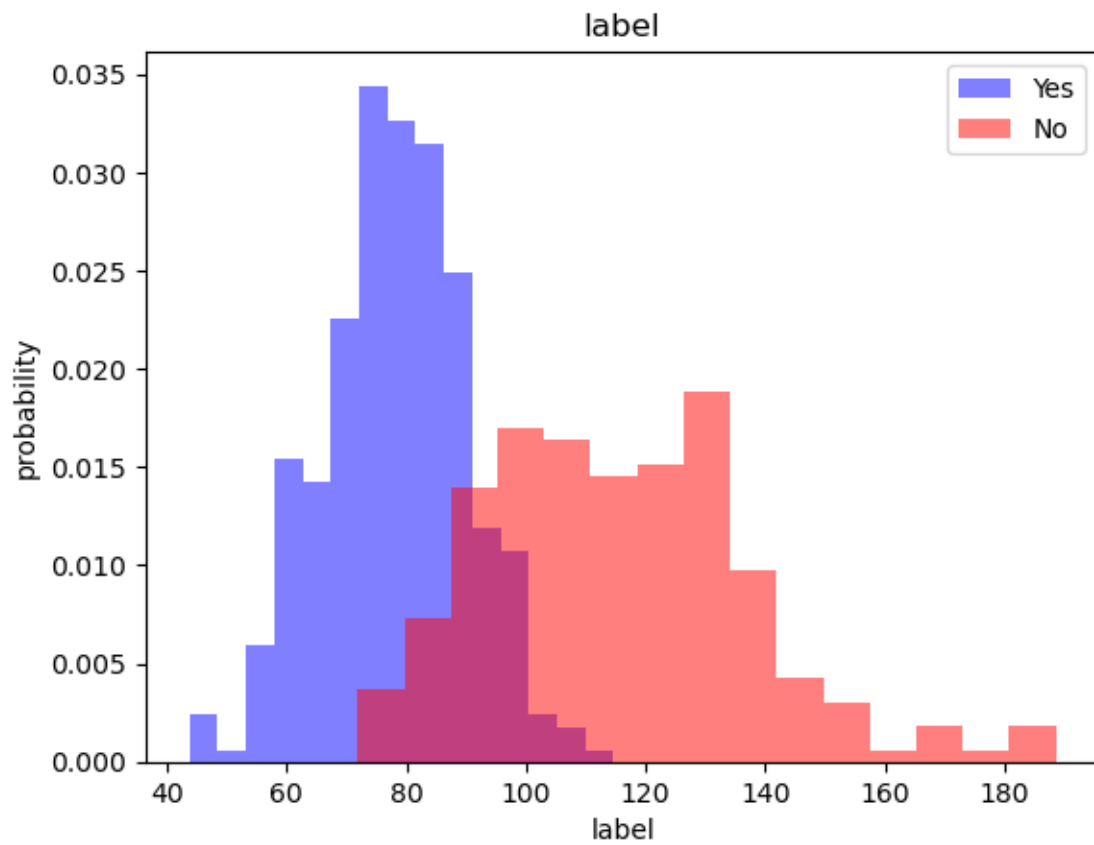
```
[14]: import matplotlib.pyplot as plt
```

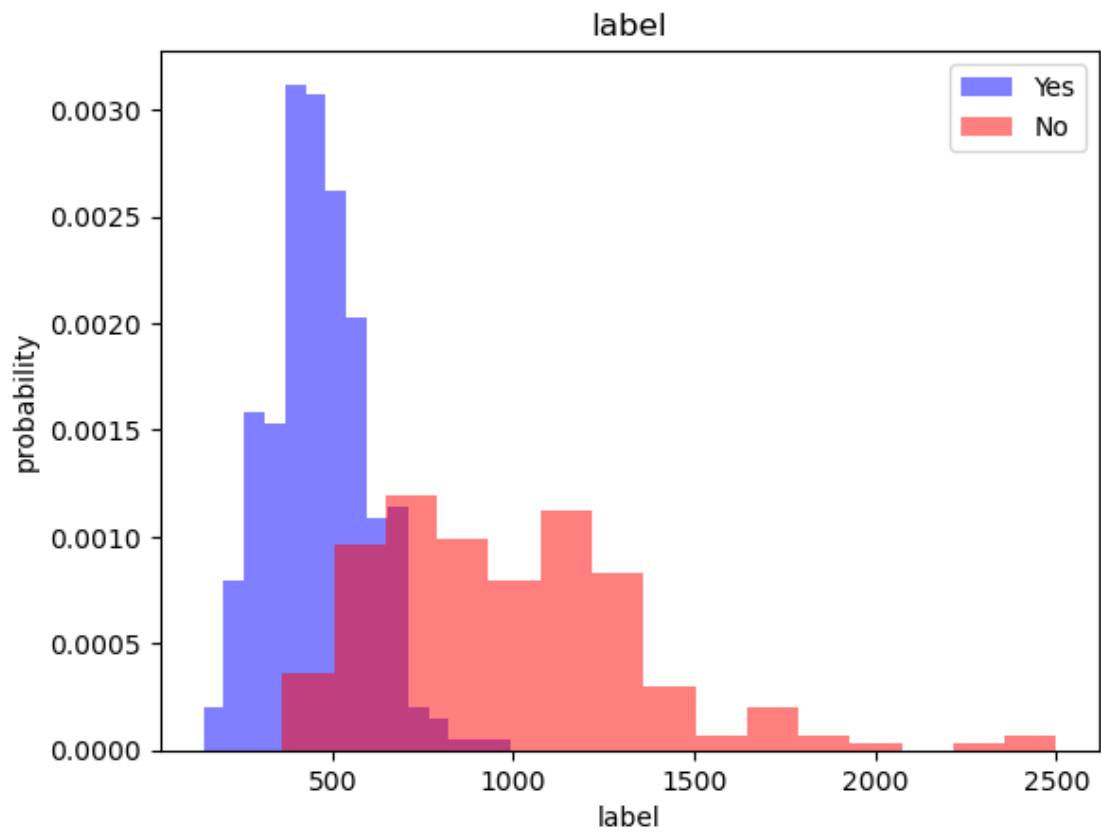
```
[15]: for i in range(len(df.columns[: -1])):
    label = df.columns[i]
    # Line 3
    plt.hist(df[df["target"] == 1][label], color="blue", label="Yes", alpha=0.
↳ 5, density=True, bins=15)

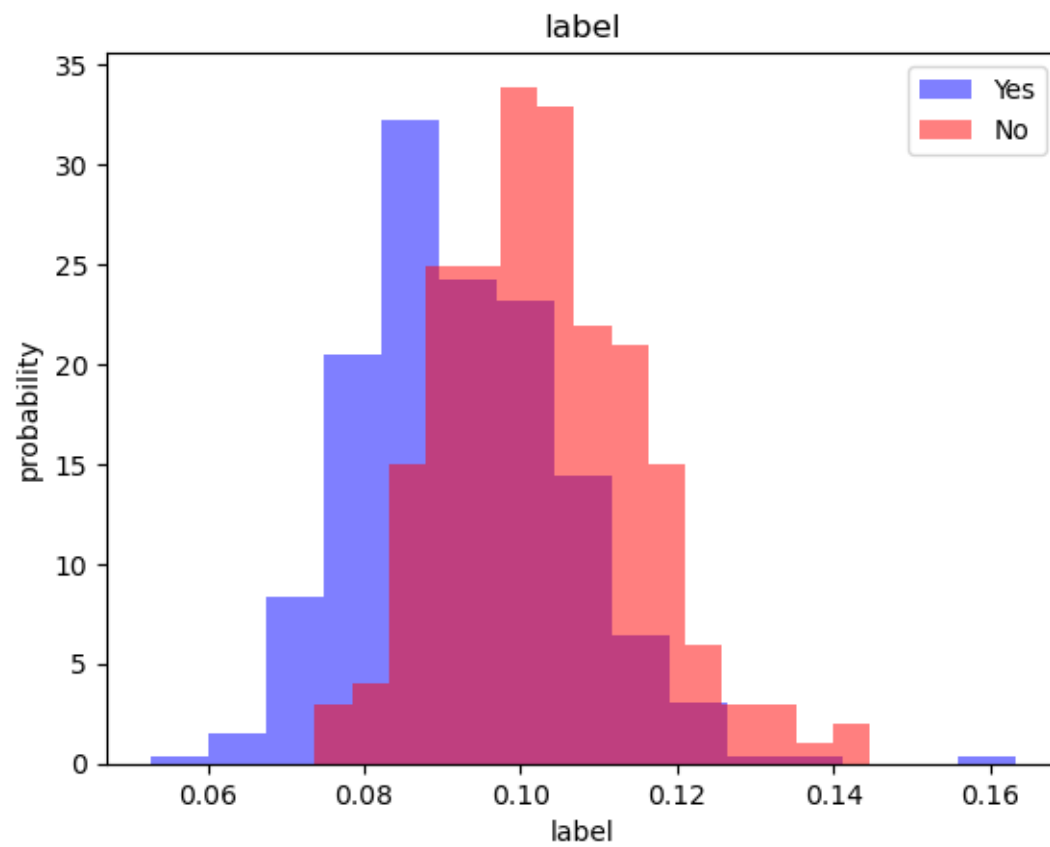
    # Line 4 (This line will likely have the same error)
    plt.hist(df[df["target"] == 0][label], color="red", label="No", alpha=0.5,
↳ density=True, bins=15)
    plt.title("label")
    plt.xlabel("label")
    plt.ylabel("probability")
    plt.legend()
    plt.show()
```

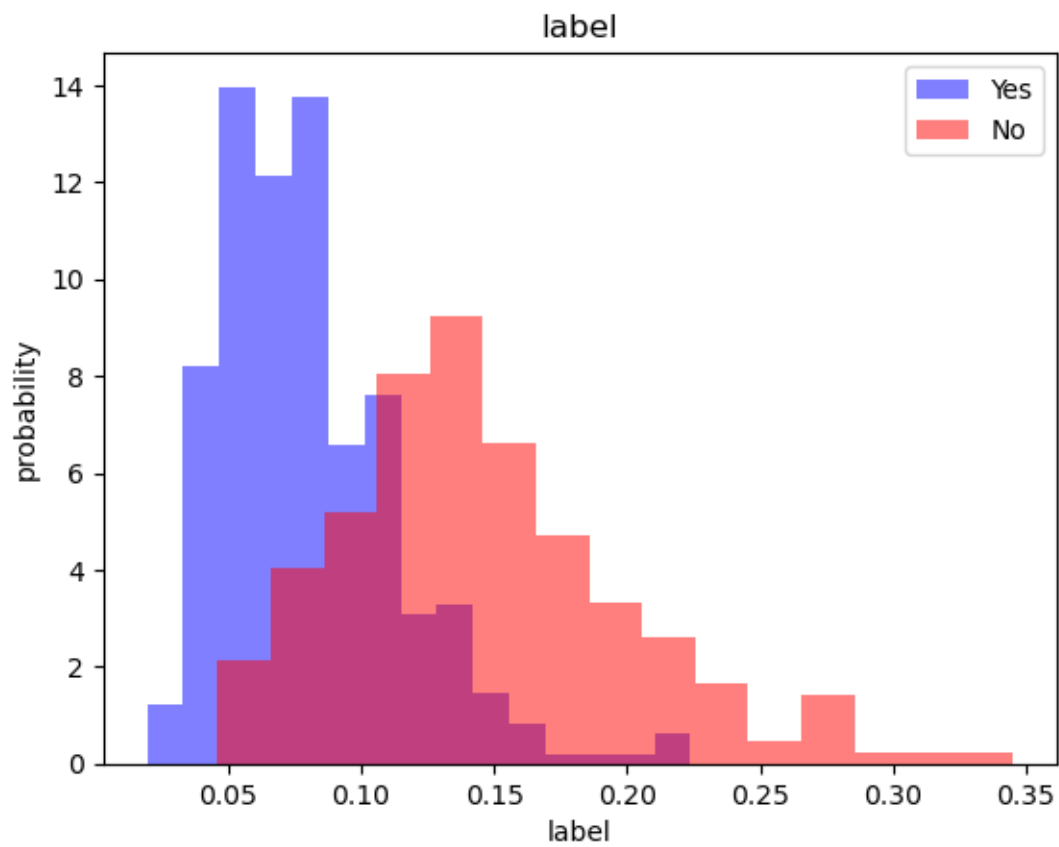


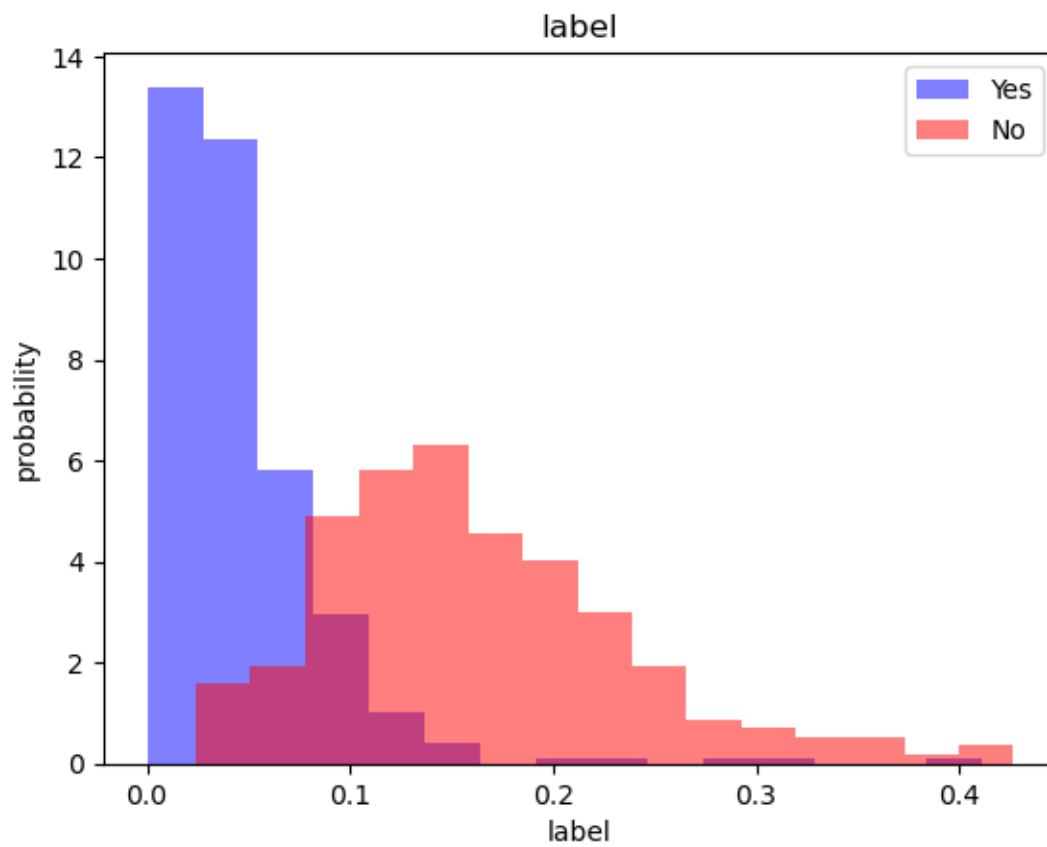


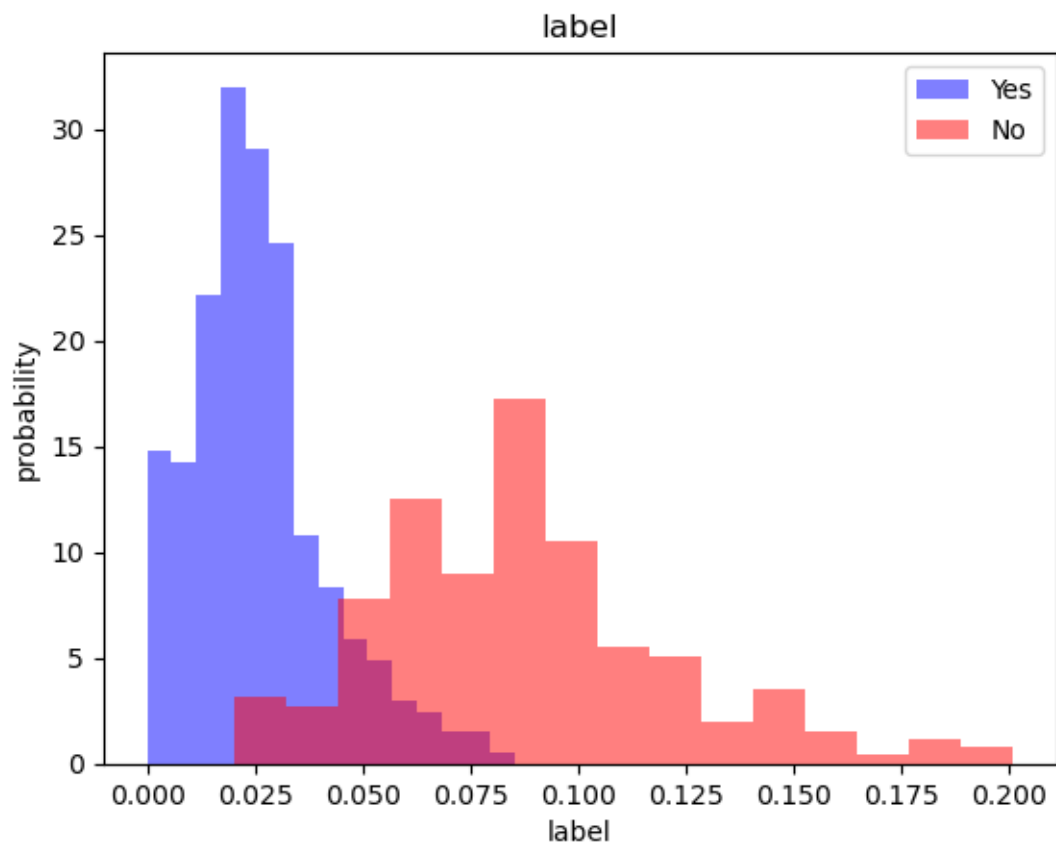


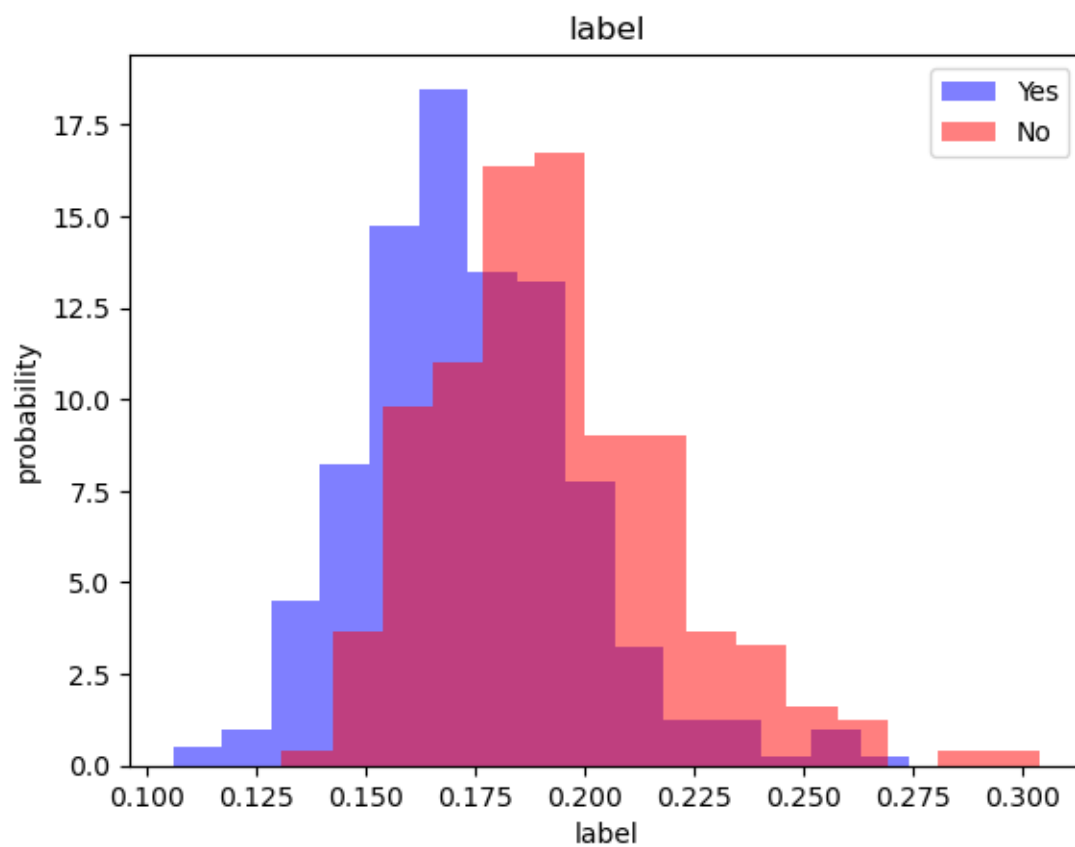


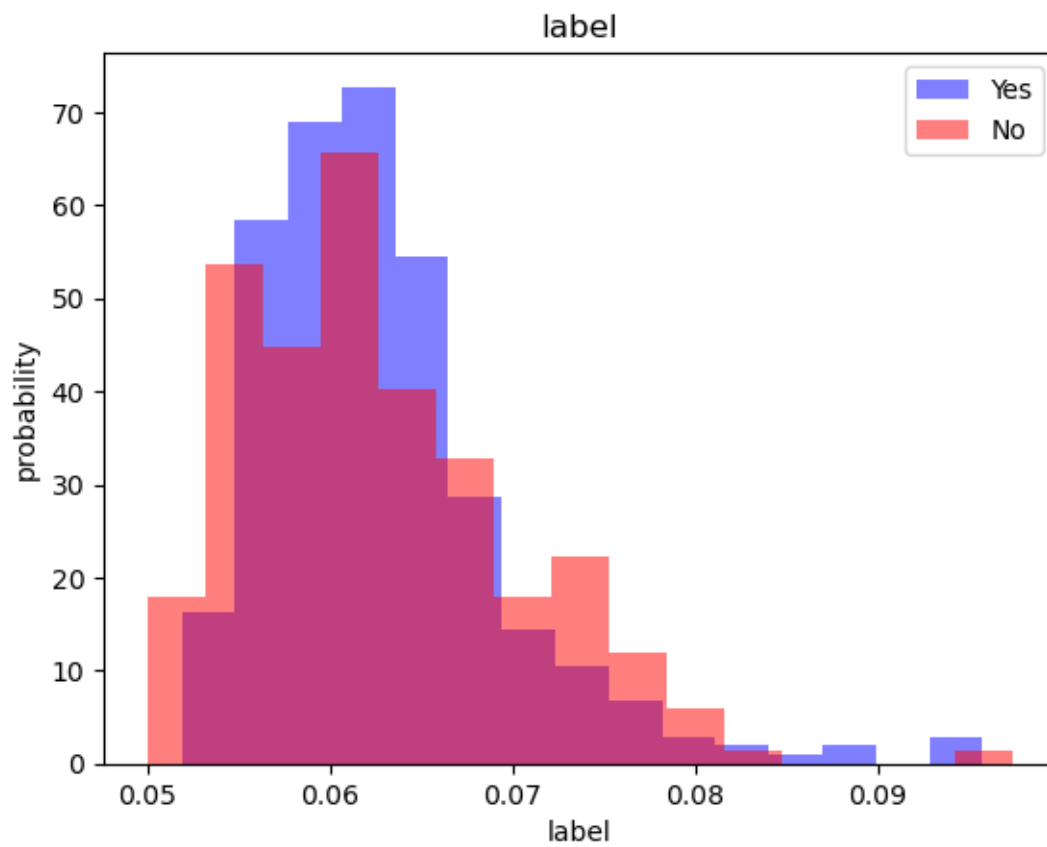


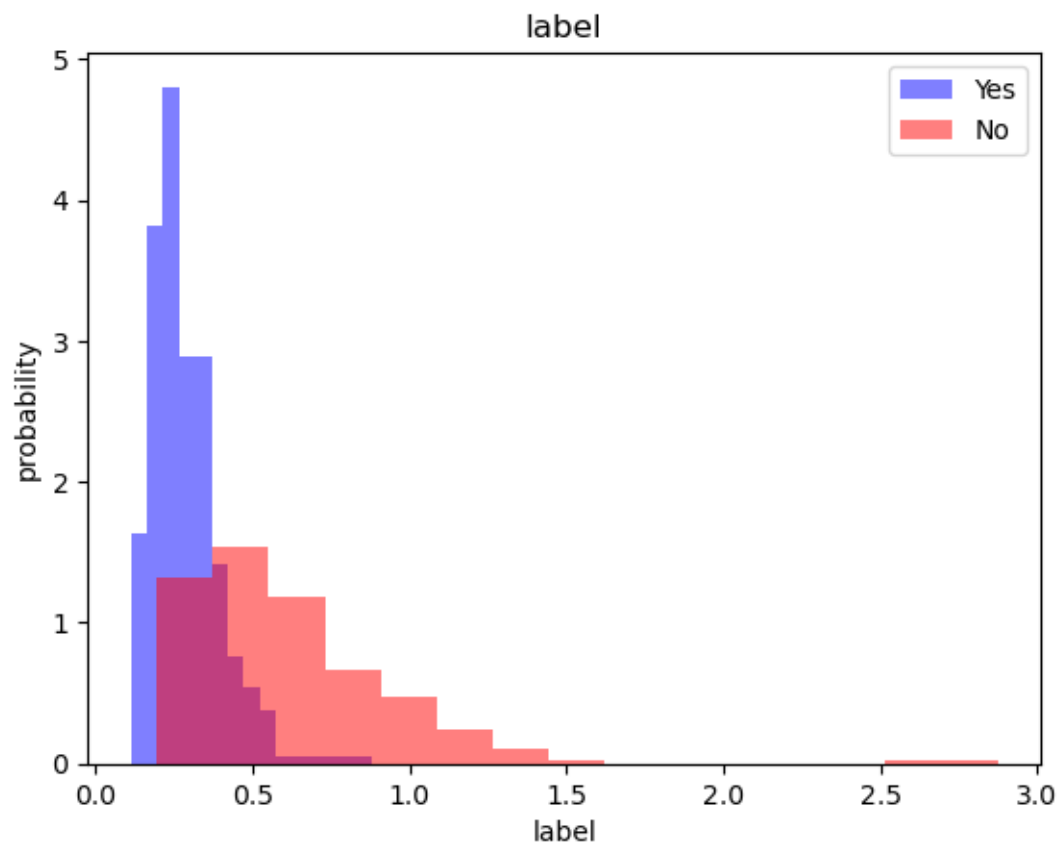


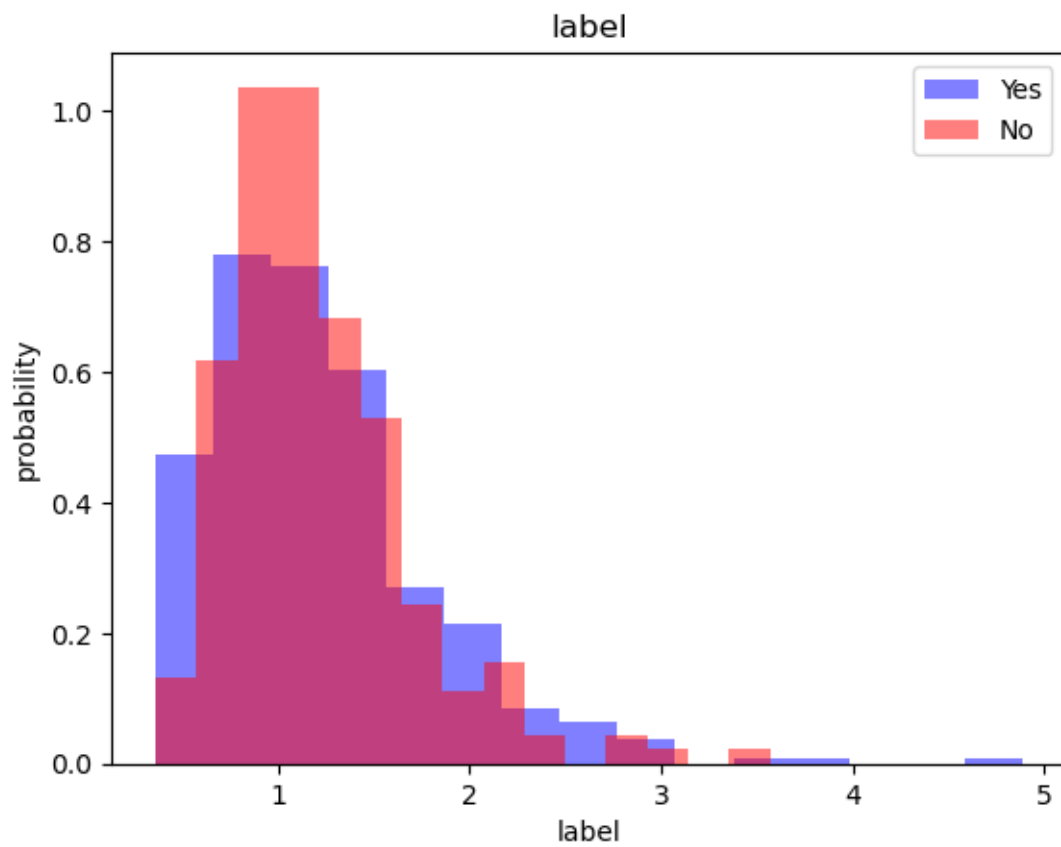


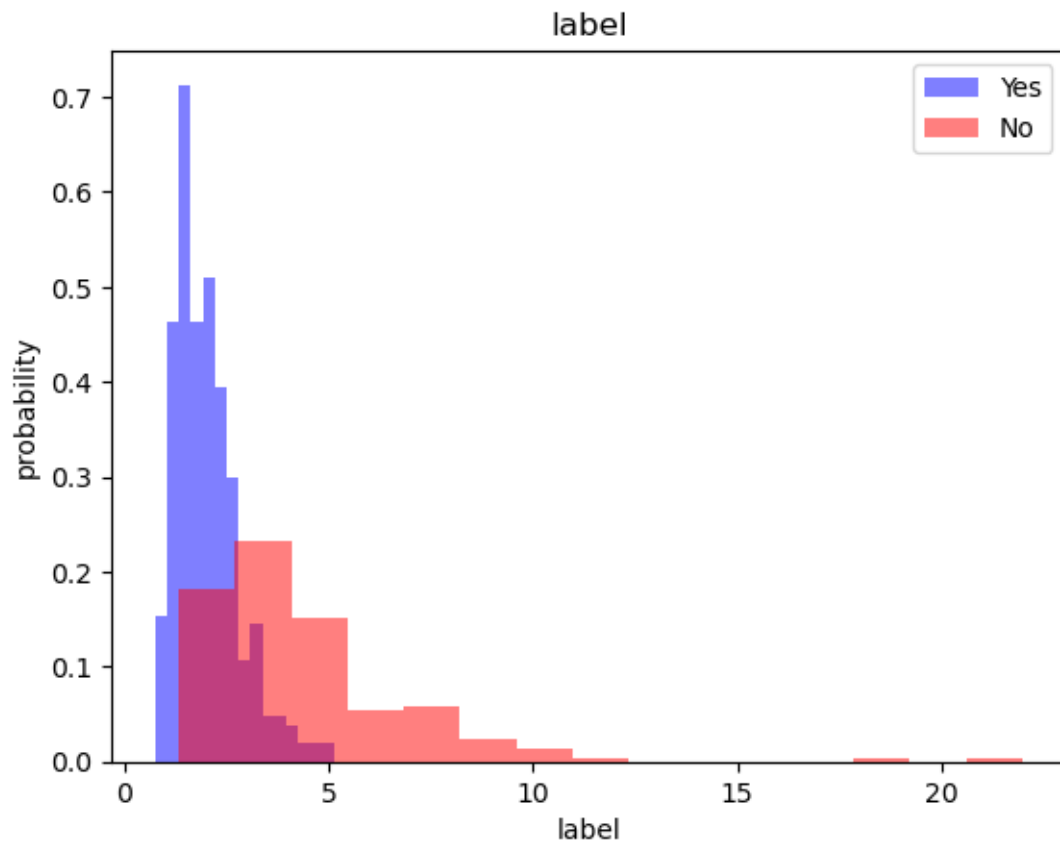


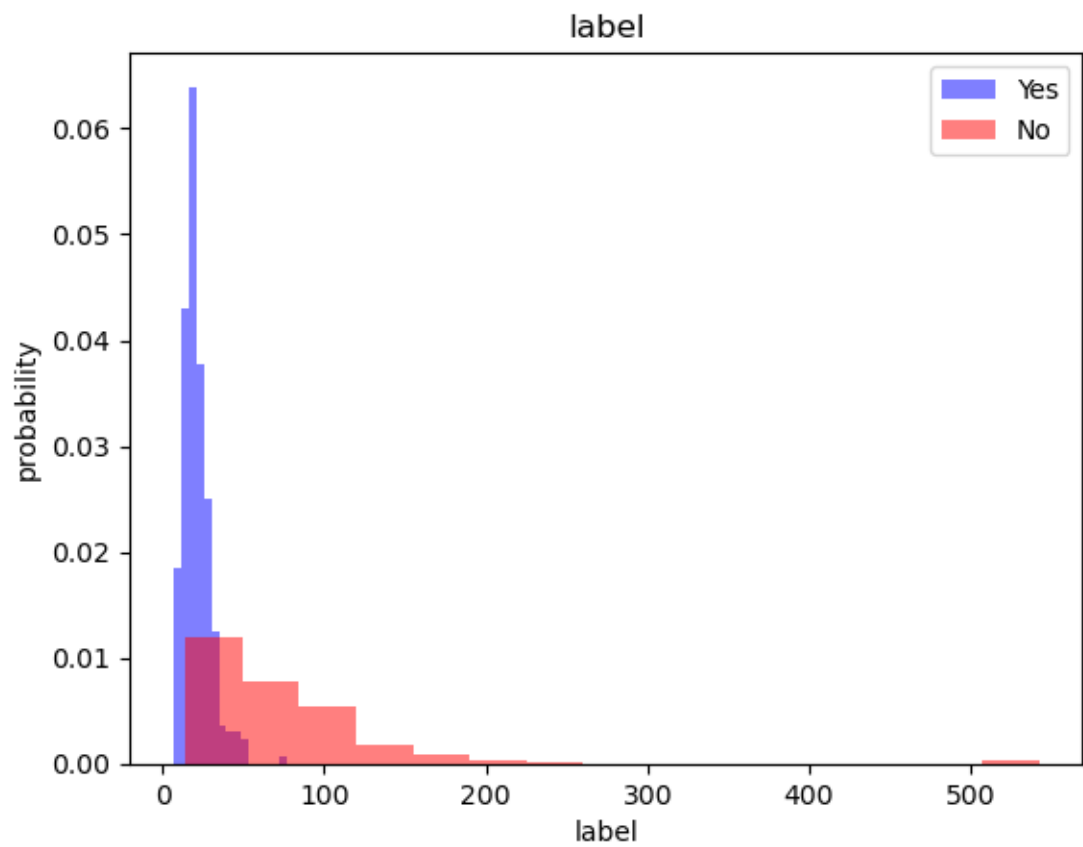


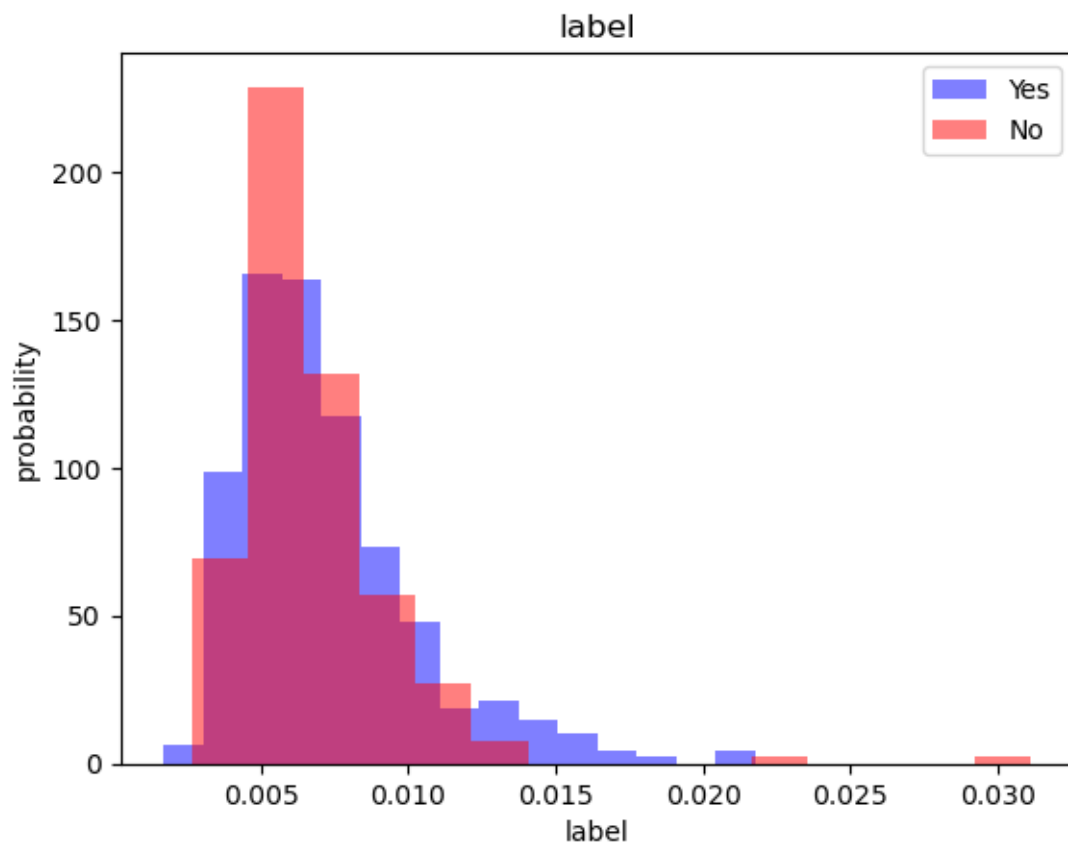


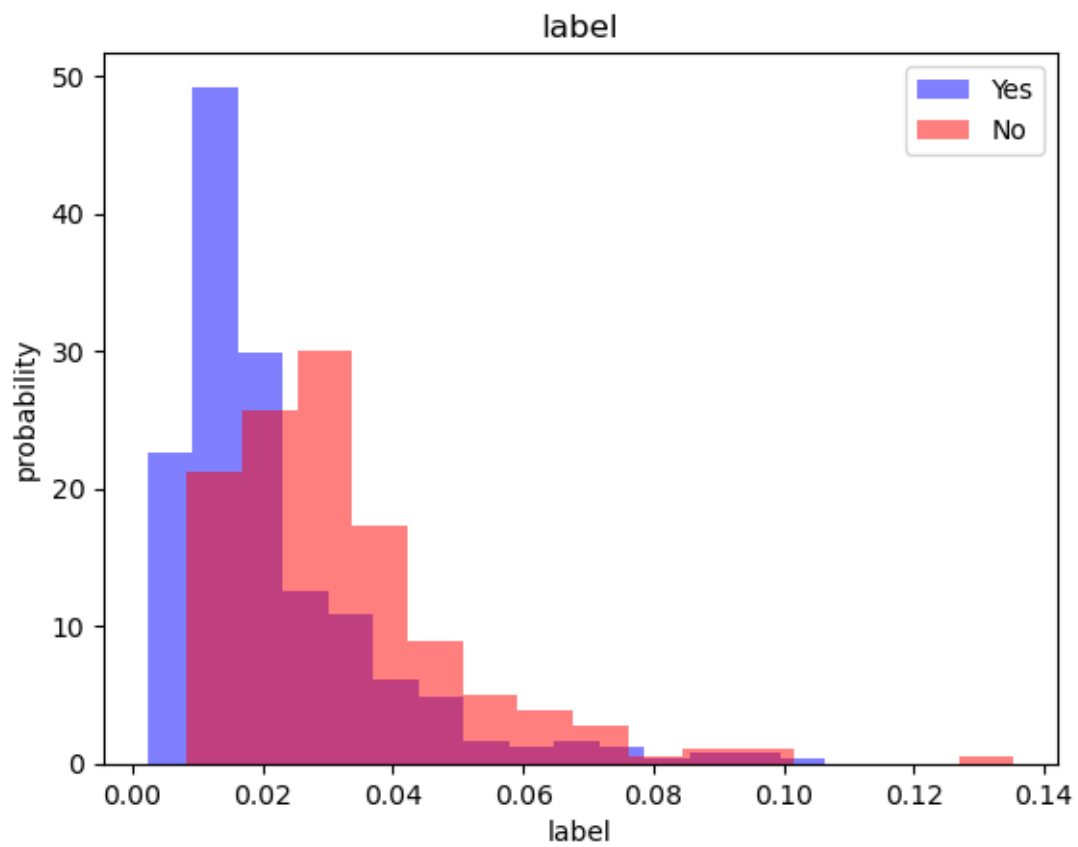


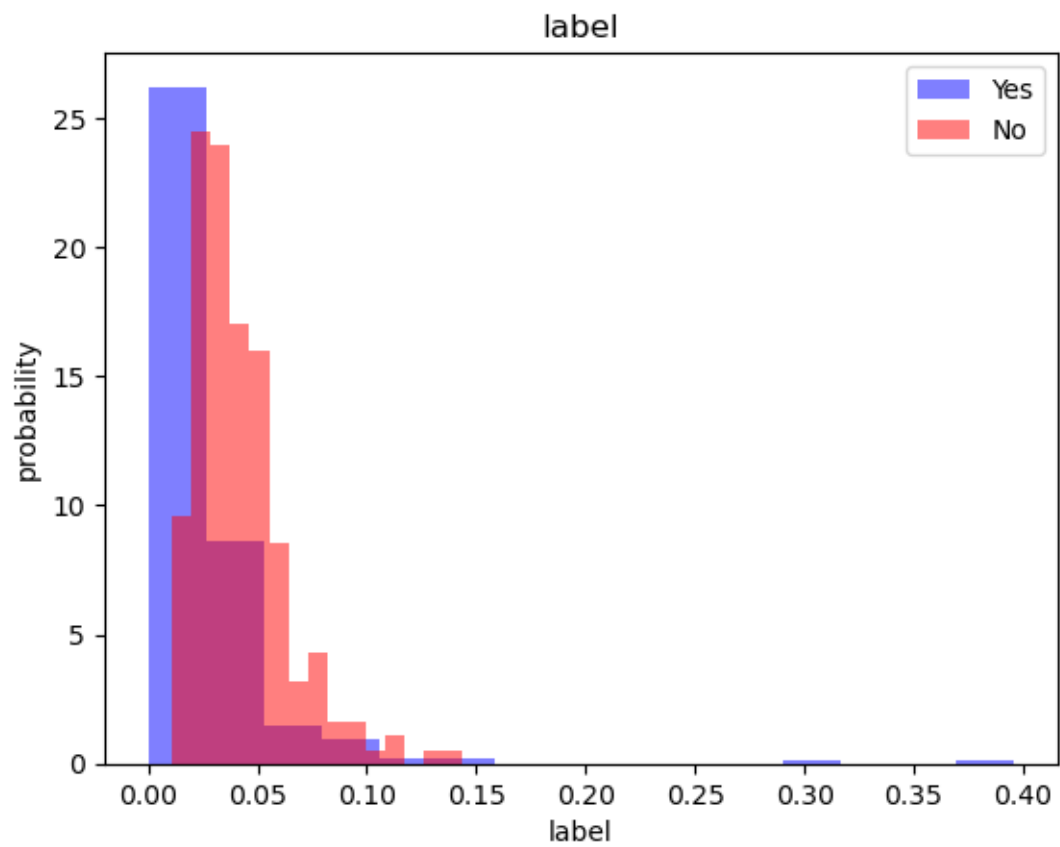


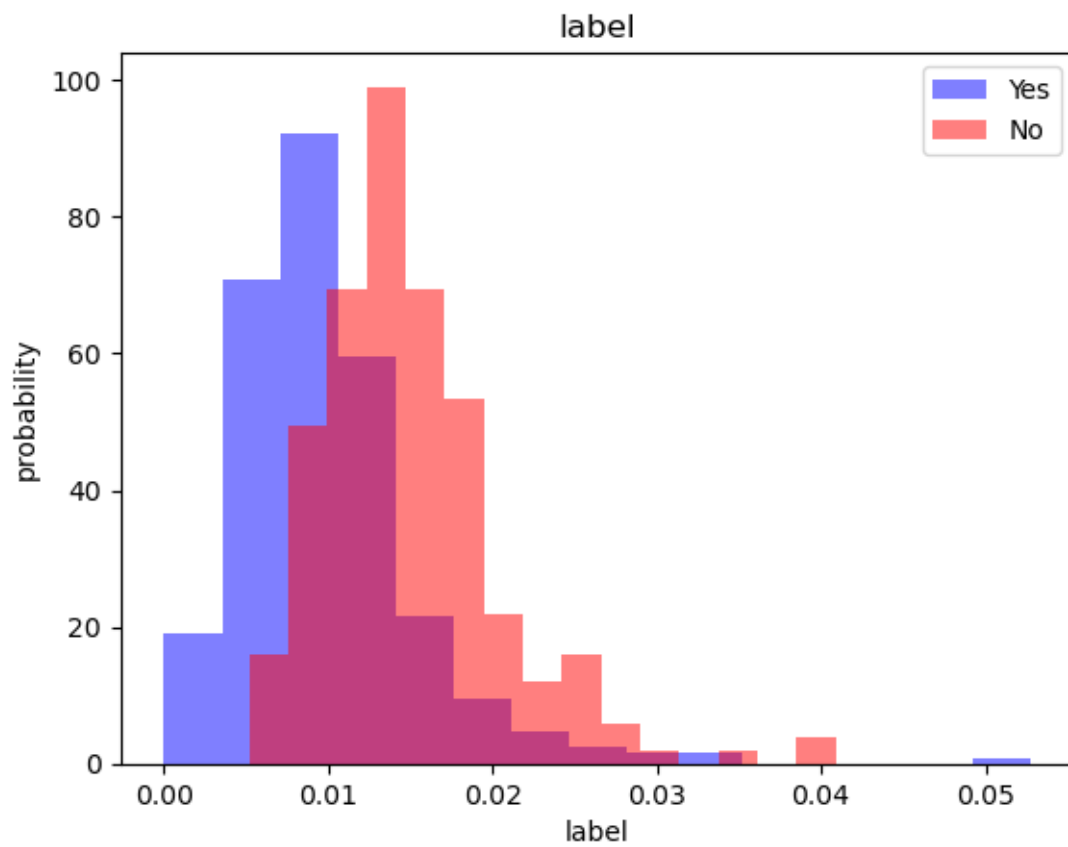


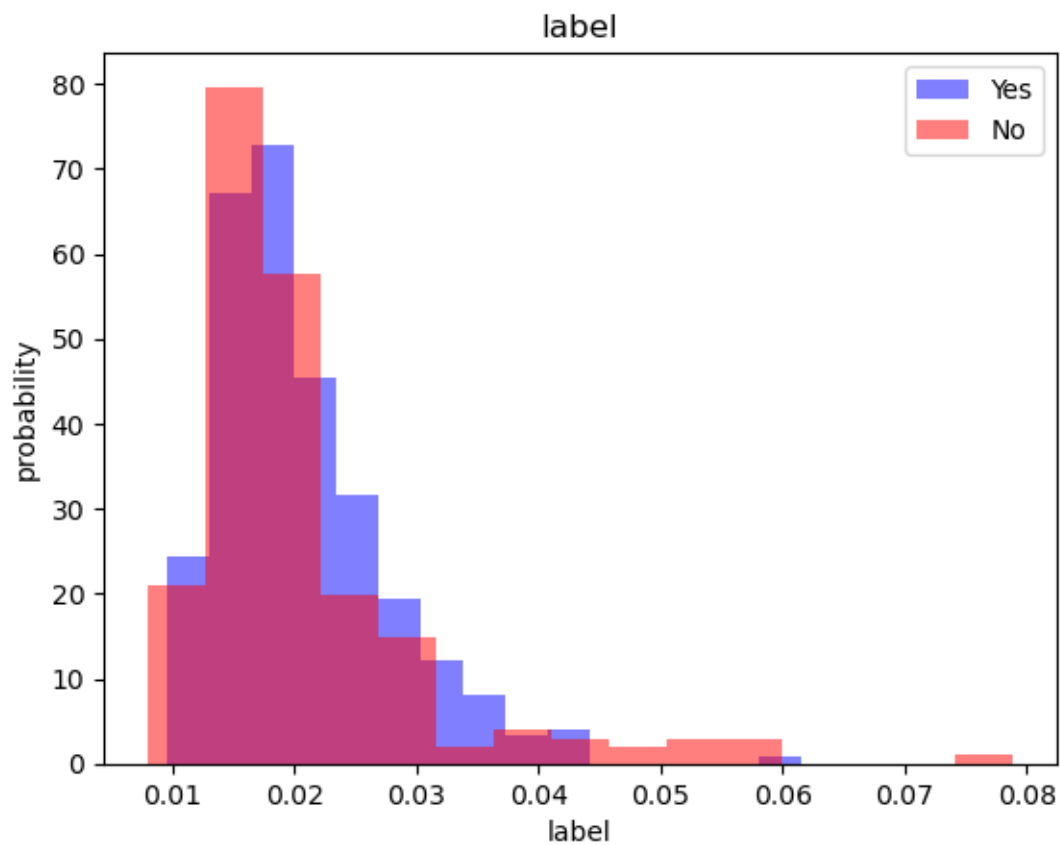


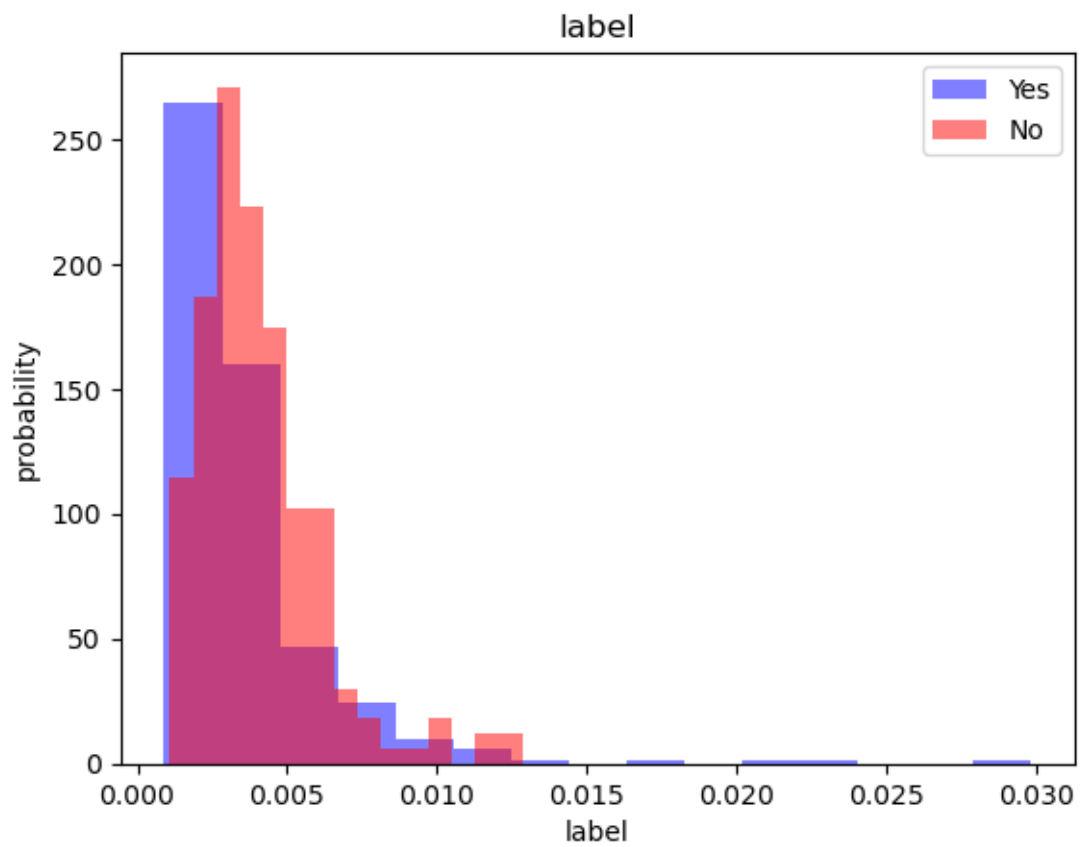


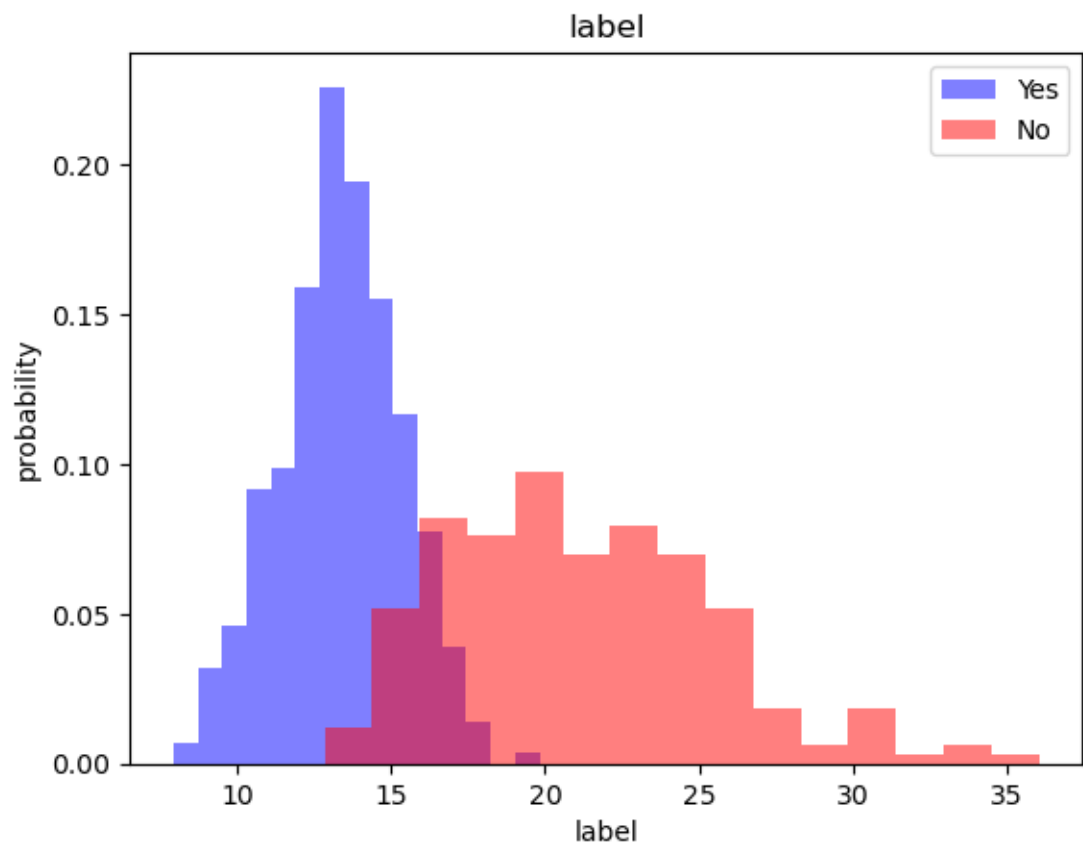


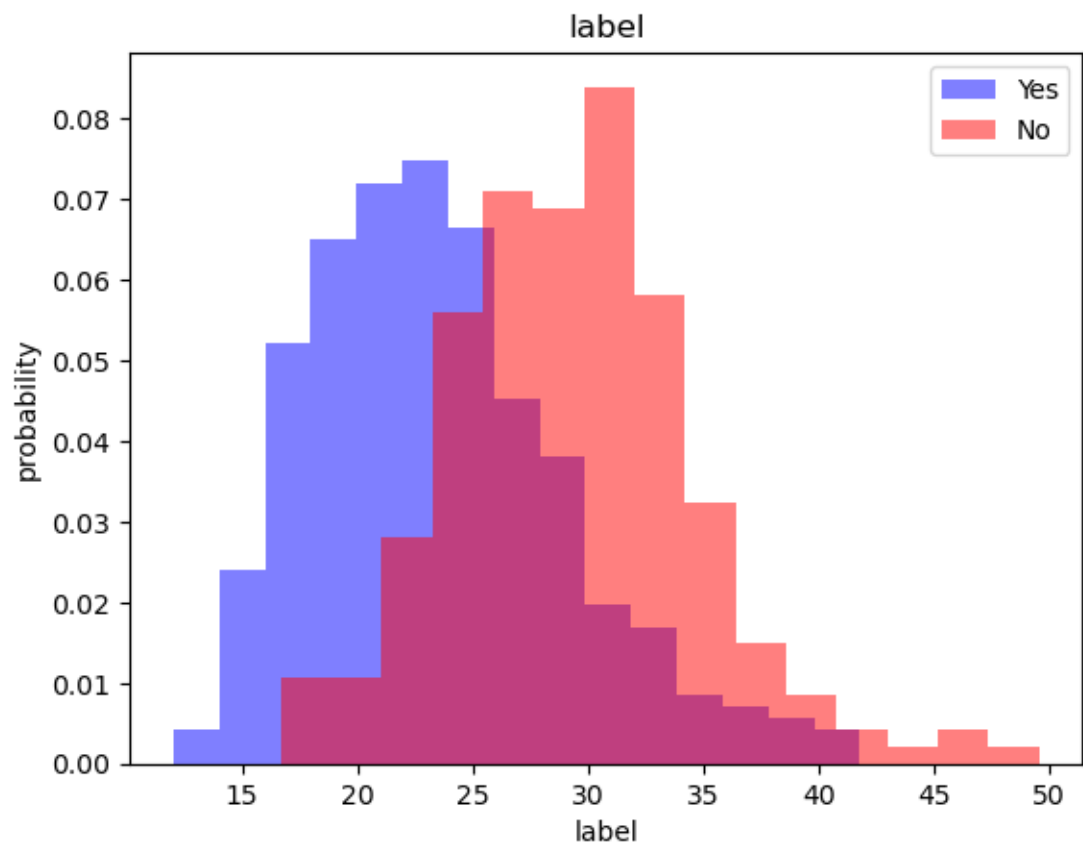


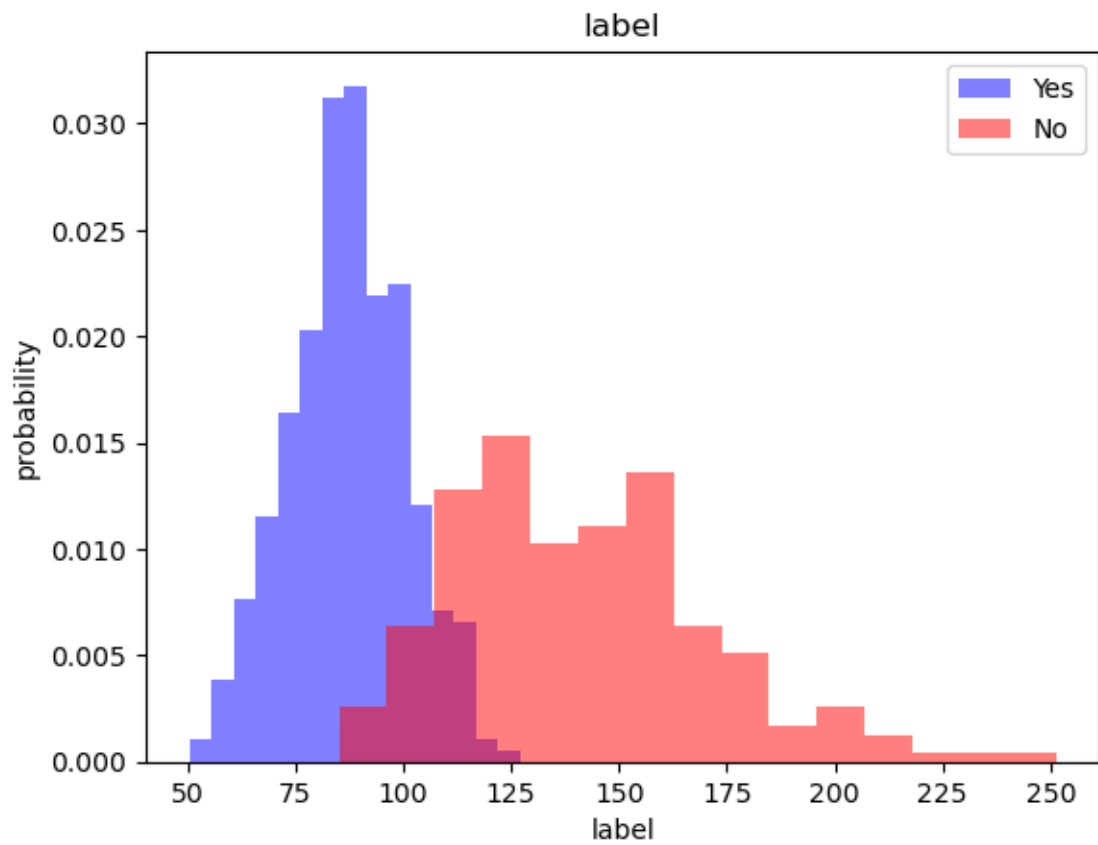


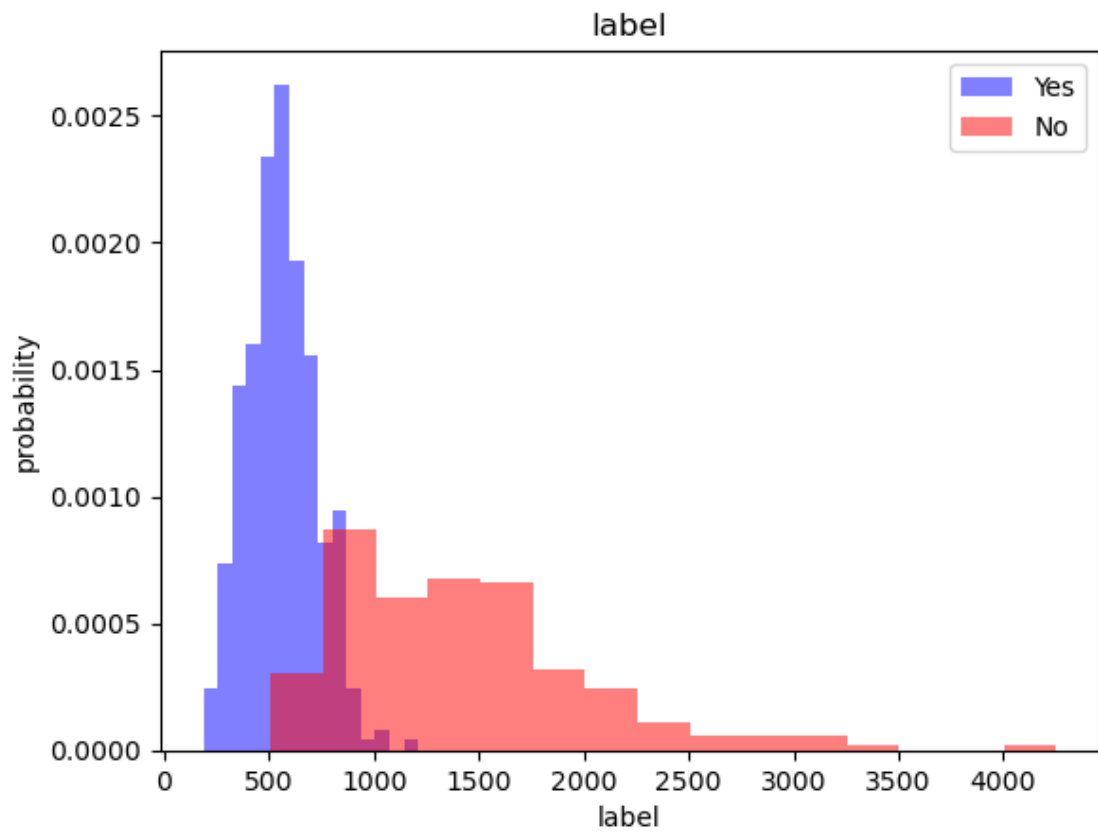


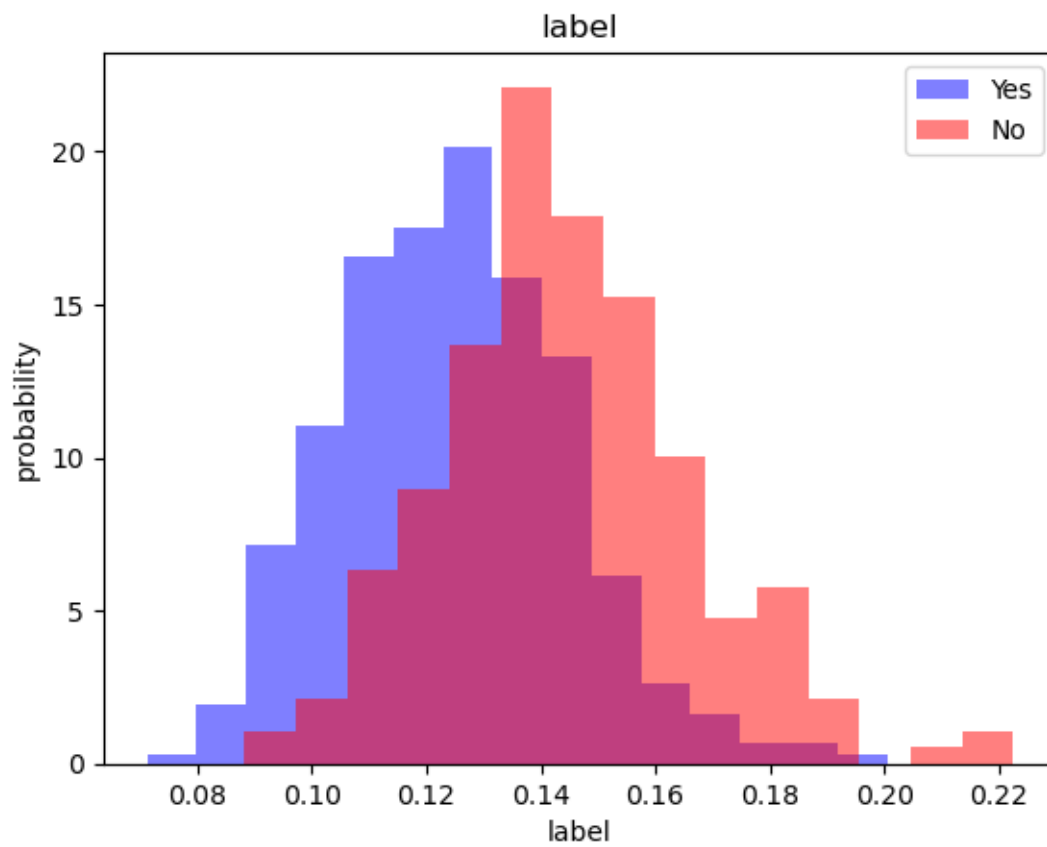


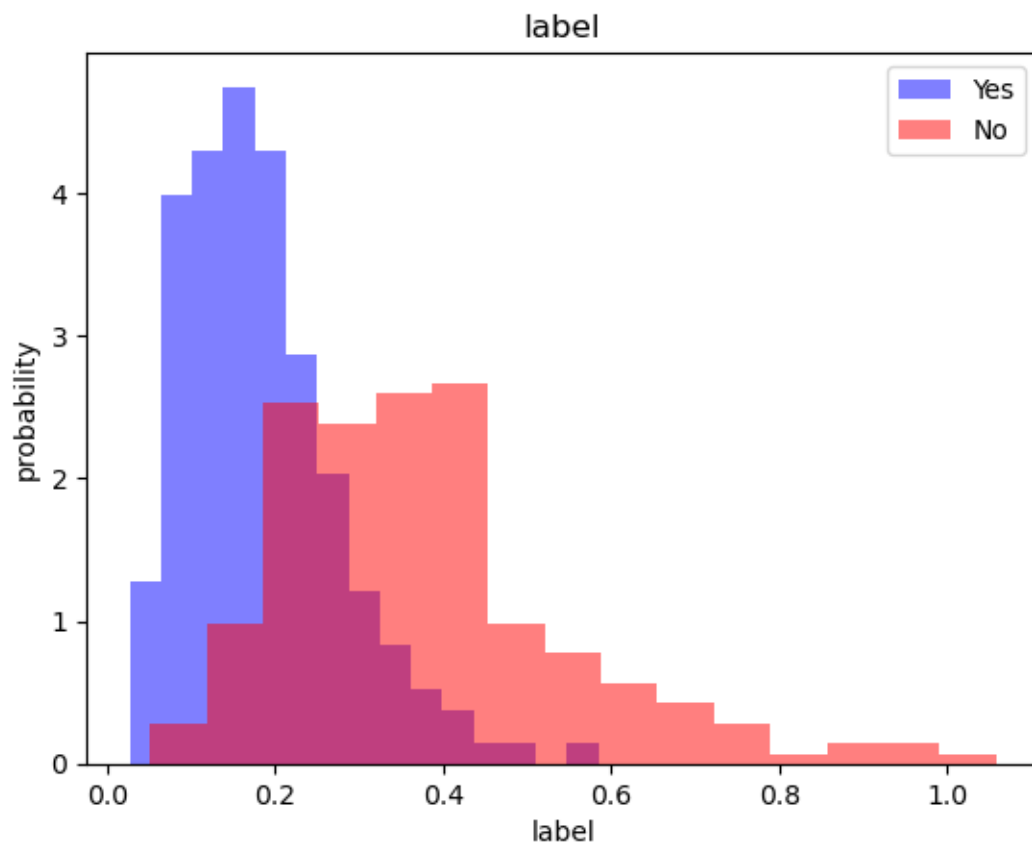


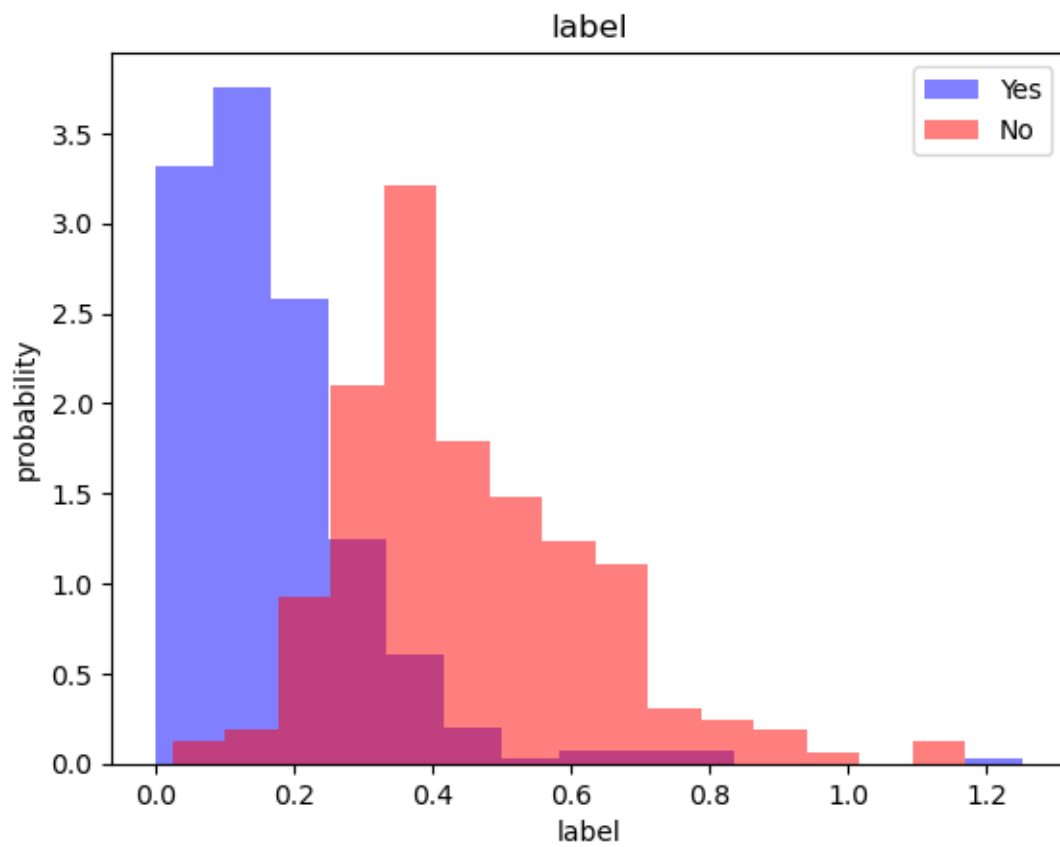


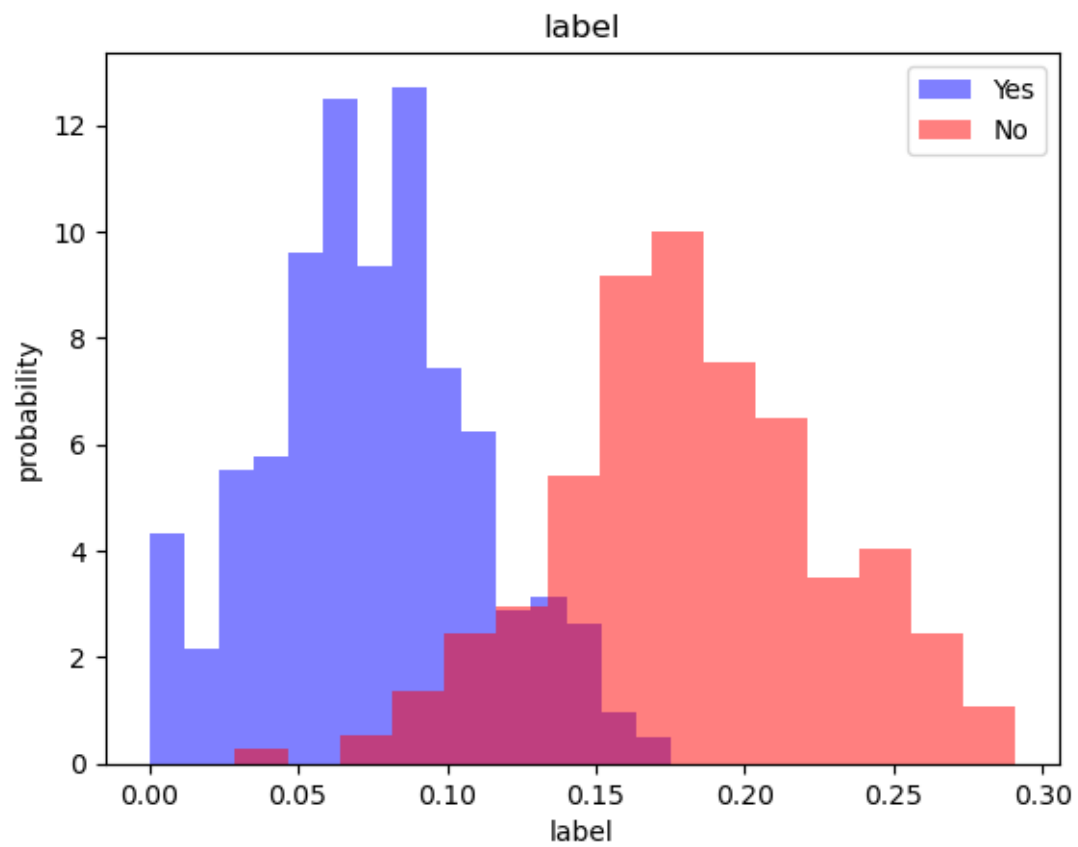


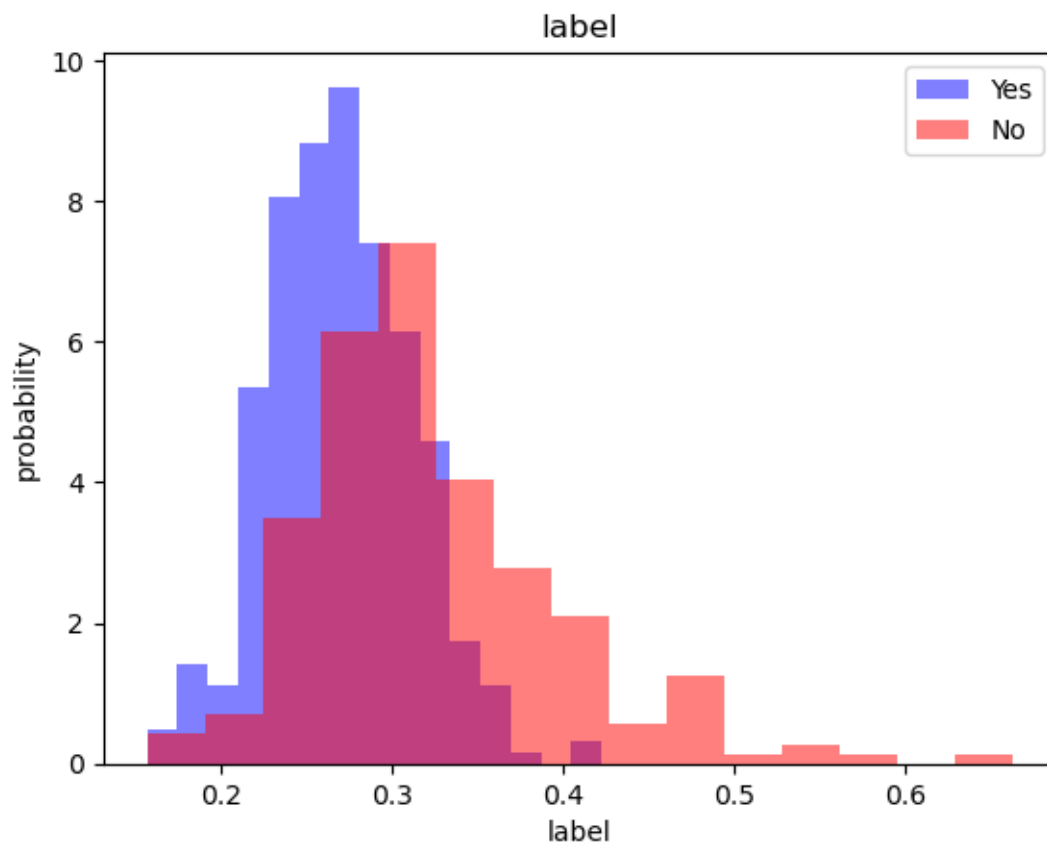


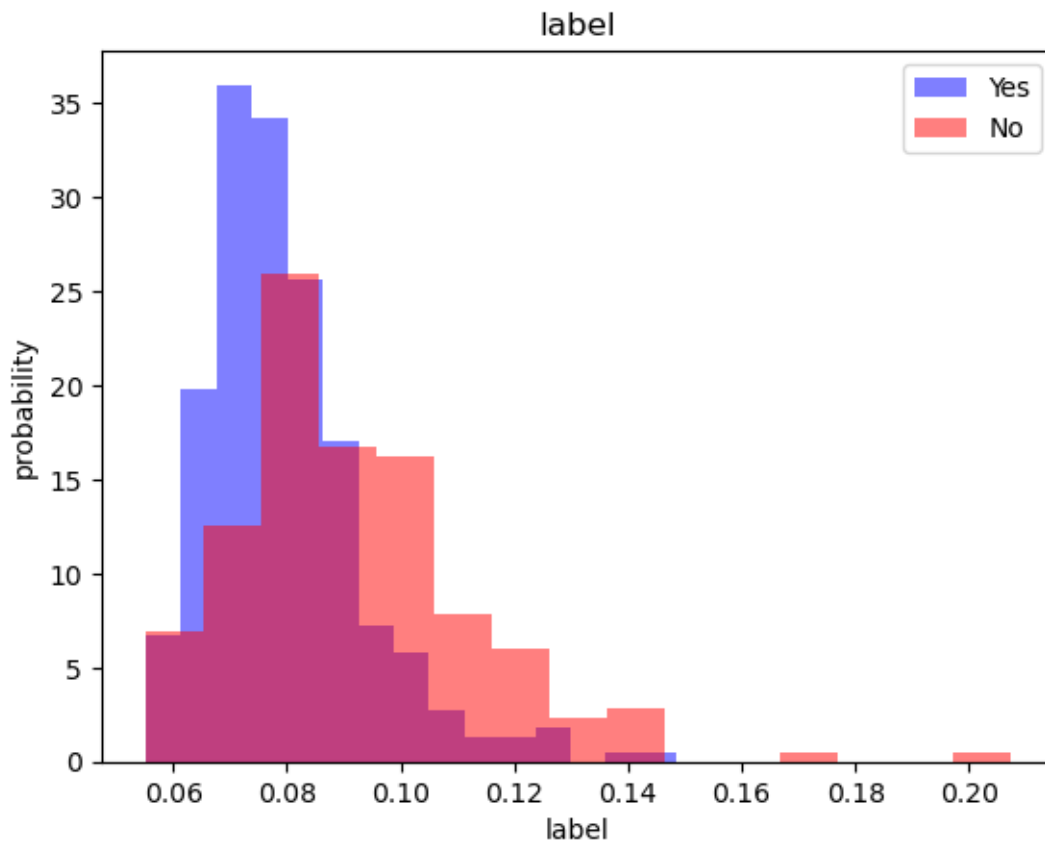












```
[16]: from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
```

```
[17]: X = df.drop( "target", axis = 1) # the Feature Variable
      y = df["target"] # Target variable contains "Yes" == 1, "No" == 0
```

```
[18]: X.shape
```

```
[18]: (569, 30)
```

```
[19]: y.shape
```

```
[19]: (569,)
```

```
[20]: X.shape
```

```
[20]: (569, 30)
```

```
[21]: X_train,X_test,y_train, y_test = train_test_split(X,
                                                         y,
```

```
test_size = 0.2,
random_state =42
)
```

[22]: X_train

```
[22]:      mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
68          9.029         17.33         58.79         250.5         0.10660
181         21.090         26.57        142.70        1311.0         0.11410
63          9.173         13.86         59.20         260.9         0.07721
248         10.650         25.22         68.01         347.0         0.09657
60         10.170         14.88         64.55         311.9         0.11340
..          ...          ...          ...          ...          ...
71          8.888         14.64         58.79         244.0         0.09783
106         11.640         18.33         75.17         412.5         0.11420
270         14.290         16.82         90.30         632.6         0.06429
435         13.980         19.62         91.12         599.5         0.10600
102         12.180         20.52         77.22         458.7         0.08013

      mean compactness  mean concavity  mean concave points  mean symmetry  \
68          0.14130         0.31300         0.04375         0.2111
181         0.28320         0.24870         0.14960         0.2395
63          0.08751         0.05988         0.02180         0.2341
248         0.07234         0.02379         0.01615         0.1897
60          0.08061         0.01084         0.01290         0.2743
..          ...          ...          ...          ...
71          0.15310         0.08606         0.02872         0.1902
106         0.10170         0.07070         0.03485         0.1801
270         0.02675         0.00725         0.00625         0.1508
435         0.11330         0.11260         0.06463         0.1669
102         0.04038         0.02383         0.01770         0.1739

      mean fractal dimension  ...  worst radius  worst texture  \
68          0.08046  ...         10.310         22.65
181         0.07398  ...         26.680         33.48
63          0.06963  ...         10.010         19.23
248         0.06329  ...         12.250         35.19
60          0.06960  ...         11.020         17.45
..          ...  ...          ...          ...
71          0.08980  ...          9.733         15.67
106         0.06520  ...         13.140         29.26
270         0.05376  ...         14.910         20.65
435         0.06544  ...         17.040         30.80
102         0.05677  ...         13.340         32.84

      worst perimeter  worst area  worst smoothness  worst compactness  \
68          65.50         324.7         0.14820         0.43650
```

181	176.50	2089.0	0.14910	0.75840
63	65.59	310.1	0.09836	0.16780
248	77.98	455.7	0.14990	0.13980
60	69.86	368.6	0.12750	0.09866
..	...	...	...	...
71	62.56	284.4	0.12070	0.24360
106	85.51	521.7	0.16880	0.26600
270	94.44	684.6	0.08567	0.05036
435	113.90	869.3	0.16130	0.35680
102	84.58	547.8	0.11230	0.08862

	worst concavity	worst concave points	worst symmetry \
68	1.25200	0.17500	0.4228
181	0.67800	0.29030	0.4098
63	0.13970	0.05087	0.3282
248	0.11250	0.06136	0.3409
60	0.02168	0.02579	0.3557
..	...	...	...
71	0.14340	0.04786	0.2254
106	0.28730	0.12180	0.2806
270	0.03866	0.03333	0.2458
435	0.40690	0.18270	0.3179
102	0.11450	0.07431	0.2694

	worst fractal dimension
68	0.11750
181	0.12840
63	0.08490
248	0.08147
60	0.08020
..	...
71	0.10840
106	0.09097
270	0.06120
435	0.10550
102	0.06878

[455 rows x 30 columns]

[23]: X_test

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
204	12.47	18.60	81.09	481.9	0.09965
70	18.94	21.31	123.60	1130.0	0.09009
131	15.46	19.48	101.70	748.9	0.10920
431	12.40	17.68	81.47	467.8	0.10540
540	11.54	14.44	74.65	402.9	0.09984

..	...	...	...	...	...
486	14.64	16.85	94.21	666.0	0.08641
75	16.07	19.65	104.10	817.7	0.09168
249	11.52	14.93	73.87	406.3	0.10130
238	14.22	27.85	92.55	623.9	0.08223
265	20.73	31.12	135.70	1419.0	0.09469

	mean compactness	mean concavity	mean concave points	mean symmetry \
204	0.10580	0.08005	0.03821	0.1925
70	0.10290	0.10800	0.07951	0.1582
131	0.12230	0.14660	0.08087	0.1931
431	0.13160	0.07741	0.02799	0.1811
540	0.11200	0.06737	0.02594	0.1818
..	...	...	...	...
486	0.06698	0.05192	0.02791	0.1409
75	0.08424	0.09769	0.06638	0.1798
249	0.07808	0.04328	0.02929	0.1883
238	0.10390	0.11030	0.04408	0.1342
265	0.11430	0.13670	0.08646	0.1769

	mean fractal dimension	...	worst radius	worst texture \
204	0.06373	...	14.97	24.64
70	0.05461	...	24.86	26.58
131	0.05796	...	19.26	26.00
431	0.07102	...	12.88	22.91
540	0.06782	...	12.26	19.68
..	...	...	...	...
486	0.05355	...	16.46	25.44
75	0.05391	...	19.77	24.56
249	0.06168	...	12.65	21.19
238	0.06129	...	15.75	40.54
265	0.05674	...	32.49	47.16

	worst perimeter	worst area	worst smoothness	worst compactness \
204	96.05	677.9	0.1426	0.2378
70	165.90	1866.0	0.1193	0.2336
131	124.90	1156.0	0.1546	0.2394
431	89.61	515.8	0.1450	0.2629
540	78.78	457.8	0.1345	0.2118
..	...	...	...	...
486	106.00	831.0	0.1142	0.2070
75	128.80	1223.0	0.1500	0.2045
249	80.88	491.8	0.1389	0.1582
238	102.50	764.0	0.1081	0.2426
265	214.00	3432.0	0.1401	0.2644

worst concavity	worst concave points	worst symmetry \
-----------------	----------------------	------------------

204	0.2671	0.10150	0.3014
70	0.2687	0.17890	0.2551
131	0.3791	0.15140	0.2837
431	0.2403	0.07370	0.2556
540	0.1797	0.06918	0.2329
..	...	...	...
486	0.2437	0.07828	0.2455
75	0.2829	0.15200	0.2650
249	0.1804	0.09608	0.2664
238	0.3064	0.08219	0.1890
265	0.3442	0.16590	0.2868

worst fractal dimension	
204	0.08750
70	0.06589
131	0.08019
431	0.09359
540	0.08134
..	...
486	0.06596
75	0.06387
249	0.07809
238	0.07796
265	0.08218

[114 rows x 30 columns]

```
[24]: y_train,y_test
```

```
[24]: (68      1
      181     0
      63      1
      248     1
      60      1
      ..
      71      1
      106     1
      270     1
      435     0
      102     1
      Name: target, Length: 455, dtype: int64,
      204     1
      70      0
      131     0
      431     1
      540     1
      ..)
```

```

486    1
75     0
249    1
238    1
265    0
Name: target, Length: 114, dtype: int64)

```

```

[25]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test)

```

```

[26]: from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(X_train_scaled, y_train)
y_pred = lr.predict(X_test_scaled)

```

```

[27]: print(y_pred)

```

```

[1 0 0 1 1 0 0 0 1 1 1 0 1 0 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 0 1 1 1 1 1 1 0
 1 0 1 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 0 0 1 1 1 0 0 1 1 0 0 1 0
 1 1 1 1 1 1 0 1 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 1 0 0 1 0 0 1 1 1 0 1 1 0
 1 1 0]

```

```

[28]: print(y.value_counts())

```

```

target
1     357
0     212
Name: count, dtype: int64

```

```

[29]: model = LinearRegression()
model.fit(X_train_scaled, y_train)

predictions = model.predict(X_test_scaled)
predictions

```

```

[29]: array([ 0.63968721,  0.22740226,  0.32361512,  1.200943   ,  1.19426351,
 -0.45850344,  0.03507188,  0.51100044,  0.34253912,  0.96568587,
  0.8023944   ,  0.37382778,  0.89913764,  0.5195883   ,  0.91616539,
  0.25508887,  0.9082683   ,  1.14003361,  1.33145512,  0.14643982,
  0.66789428,  0.86898586, -0.24725593,  1.03923885,  0.95661002,
  0.82858602,  1.03646754,  0.92965204,  0.93311782, -0.03928239,
  1.06383079,  1.0202386   ,  0.73468352,  0.89497087,  1.02367197,
  0.80975679,  0.49603914,  0.57409927,  0.29801476,  0.76982672,
  1.1605838   ,  0.11940268,  0.97874026,  1.01047958,  0.50803949,
  0.66681999,  0.85480572,  0.96623662,  0.81058351,  1.17373593,
  0.24346832,  0.05803547,  0.58223574,  0.63661059,  1.0391142   ,
  0.79723021,  0.9755658   , -0.17773385,  0.60053297,  1.02399552,

```

```

0.95398461, -0.10452604, -0.22631651, 0.83895359, 0.87293584,
0.63006452, -0.03840662, 0.06758752, 0.96062681, 0.89850115,
0.14113892, 0.15386398, 0.84481598, 0.37501308, 0.92137147,
0.82740334, 0.78402196, 0.61992903, 1.18592802, 0.74912319,
0.44056318, 1.01108601, 0.61502411, 0.11565107, 0.35401415,
0.22506261, -0.17239285, 0.20463834, 1.05970704, 0.9494094 ,
0.82553472, 0.78431158, 0.82323408, 1.07999724, 0.92211558,
0.94742323, 0.0980465 , -0.16009551, 0.99463806, 0.24787306,
0.28363121, 1.21732617, 0.31330849, 0.20591948, 0.73814434,
0.72881162, 0.83685141, -0.01237638, 0.74713578, 0.77607129,
0.20715039, 0.96434081, 0.65350327, 0.21373627])

```

```

[30]: residual = y_test - predictions
      residual

```

```

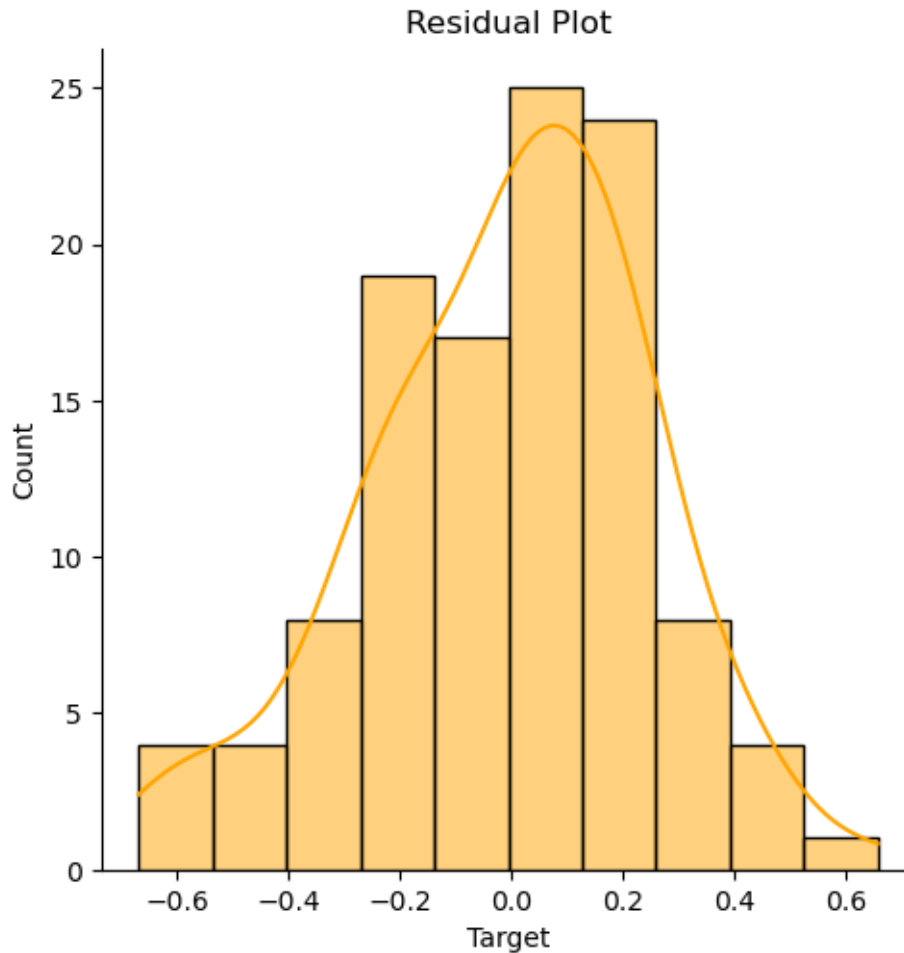
[30]: 204    0.360313
      70    -0.227402
      131   -0.323615
      431   -0.200943
      540   -0.194264
      ...
      486    0.223929
      75    -0.207150
      249    0.035659
      238    0.346497
      265   -0.213736
      Name: target, Length: 114, dtype: float64

```

```

[31]: sns.displot(residual, kde =True, bins =10, color = "orange")
      plt.title("Residual Plot")
      plt.xlabel("Target")
      plt.ylabel("Count")
      plt.show()

```



```
[32]: from sklearn.metrics import mean_squared_error, mean_absolute_error
import math
```

```
[33]: print (f"Mean Squared Errorr: {mean_squared_error(y_test,y_pred)}")
print (f"Mean Absolute Error : {mean_absolute_error(y_test,y_pred)}")
print (f"Mean Sqrt Abslure Errorr: {math.
↪sqrt(mean_absolute_error(y_test,y_pred))}")
```

```
Mean Squared Errorr: 0.017543859649122806
Mean Absolute Error : 0.017543859649122806
Mean Sqrt Abslure Errorr: 0.13245323570650439
```

```
[34]: print("::::::::::::SUPPORT VECTOR MACHINE (SVM)::::::::::::")
↪print(" ")
```

```
::::::::::::SUPPORT VECTOR MACHINE (SVM)::::::::::::
SUPPORT VECTOR MACHINE (SVM)
```

```
[35]: from sklearn.svm import SVC
```

```
svm = SVC(kernel= 'rbf',  
           C = 100,  
           gamma='scale',  
           random_state= 42)
```

```
svm.fit(X_trained_scaled, y_train)
```

```
svm_prediction = svm.predict(X_test_scaled)
```

```
print(svm_prediction)
```

```
[1 0 0 1 1 0 0 0 0 1 1 0 1 0 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1 0  
 1 0 1 1 0 1 1 0 0 1 1 1 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 1 1 0 0 1 1 0 0 1 0  
 1 1 1 1 1 1 0 1 0 0 0 0 0 0 1 1 1 0 1 1 1 1 0 0 1 0 0 1 0 0 1 1 1 0 1 1 0  
 1 1 0]
```

```
[36]: data = {  
      'Actual_values' : y_test,  
      'Svm_Predicted_value' :svm_prediction  
    }
```

```
data = pd.DataFrame(data) # creating the dataframe containing the actual values  
    ↪and predictions
```

```
print(data)
```

	Actual_values	Svm_Predicted_value
204	1	1
70	0	0
131	0	0
431	1	1
540	1	1
..	...	...
486	1	1
75	0	0
249	1	1
238	1	1
265	0	0

```
[114 rows x 2 columns]
```

```
[37]: from sklearn.metrics import accuracy_score,  
    ↪confusion_matrix,ConfusionMatrixDisplay
```

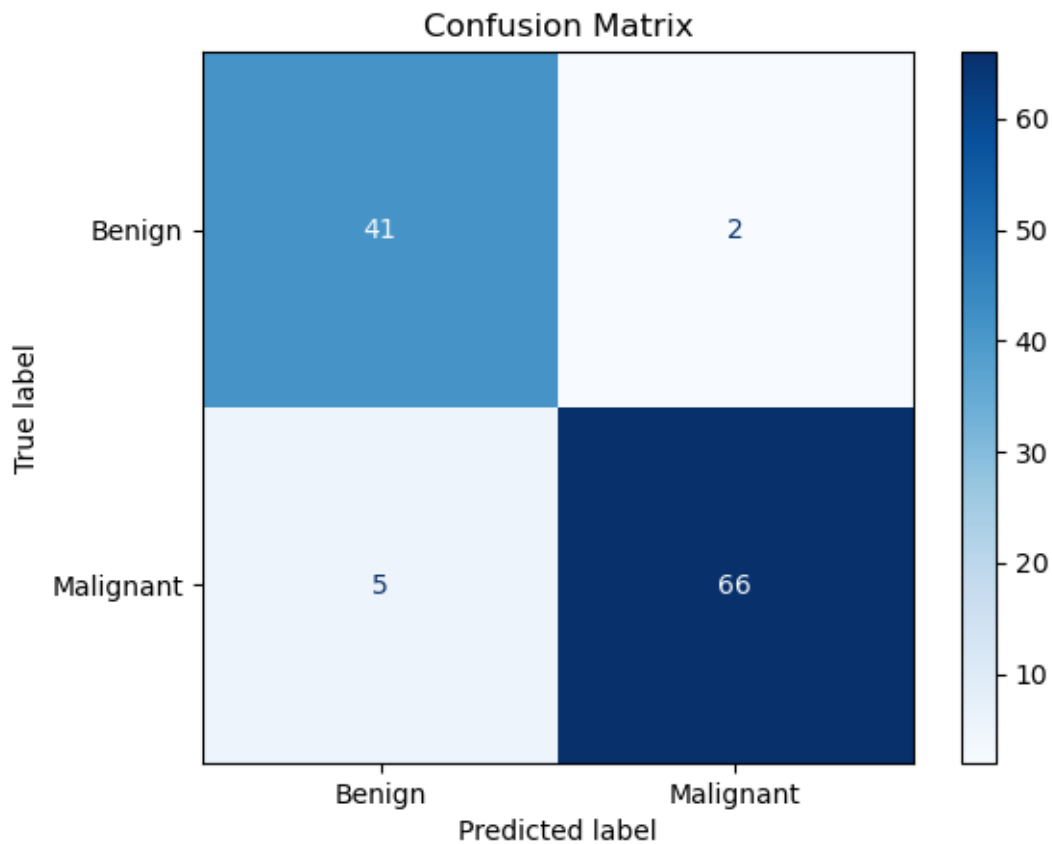
```
[38]: accuracy = accuracy_score(y_test,svm_prediction) # Evaluating the accuracy
      ↪score using the SVM
      print(f"Accuracy Score for the Support Vector Machine is : {accuracy}")
```

Accuracy Score for the Support Vector Machine is : 0.9385964912280702

```
[39]: cm = confusion_matrix(y_test, svm_prediction)
      # Display confusion matrix
      disp = ConfusionMatrixDisplay(
          confusion_matrix=cm,
          display_labels=["Benign", "Malignant"]
      )

      disp.plot(cmap="Blues")

      plt.title("Confusion Matrix")
      plt.show()
```



```
[40]: from sklearn.ensemble import RandomForestClassifier
```

```

rfc = RandomForestClassifier(n_estimators = 100, max_depth = 5, random_state = 42, n_jobs = -1)
rfc.fit(X_trained_scaled, y_train)
rfc_pred = rfc.predict(X_test_scaled)
rfc_pred

```

```

[40]: array([1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1,
          0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1,
          1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
          0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0,
          1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1,
          0, 1, 1, 0])

```

```

[41]: from boruta import BorutaPy

boruta = BorutaPy(estimator = rfc,
                  n_estimators = "auto",
                  random_state = 42)
boruta.fit(X_trained_scaled, y_train.values)

```

```

[41]: BorutaPy(estimator=RandomForestClassifier(max_depth=5, n_estimators=152,
                                                n_jobs=-1,
                                                random_state=RandomState(MT19937) at
0x20521E13A40),
              n_estimators='auto',
              random_state=RandomState(MT19937) at 0x20521E13A40)

```

```

[42]: selected_features= X.columns[boruta.support_]
print(f"Confirmed Features: {selected_features}")

```

```

Confirmed Features: Index(['mean radius', 'mean texture', 'mean perimeter',
'mean area',
      'mean smoothness', 'mean compactness', 'mean concavity',
      'mean concave points', 'mean fractal dimension', 'radius error',
      'perimeter error', 'area error', 'compactness error', 'concavity error',
      'concave points error', 'fractal dimension error', 'worst radius',
      'worst texture', 'worst perimeter', 'worst area', 'worst smoothness',
      'worst compactness', 'worst concavity', 'worst concave points',
      'worst symmetry', 'worst fractal dimension'],
      dtype='object')

```

```

[43]: feature_results = pd.DataFrame({
      "Feature": X.columns,
      "Ranking": boruta.ranking_,
      "Confirmed": boruta.support_,
      "Rejected" : ~ boruta.support_,
      "Tentative": boruta.support_weak_
    })

```

```
feature_results.sort_values("Ranking")
```

```
[43]:
```

	Feature	Ranking	Confirmed	Rejected	Tentative
0	mean radius	1	True	False	False
27	worst concave points	1	True	False	False
26	worst concavity	1	True	False	False
25	worst compactness	1	True	False	False
24	worst smoothness	1	True	False	False
23	worst area	1	True	False	False
22	worst perimeter	1	True	False	False
21	worst texture	1	True	False	False
20	worst radius	1	True	False	False
19	fractal dimension error	1	True	False	False
17	concave points error	1	True	False	False
16	concavity error	1	True	False	False
15	compactness error	1	True	False	False
28	worst symmetry	1	True	False	False
29	worst fractal dimension	1	True	False	False
12	perimeter error	1	True	False	False
10	radius error	1	True	False	False
9	mean fractal dimension	1	True	False	False
7	mean concave points	1	True	False	False
6	mean concavity	1	True	False	False
5	mean compactness	1	True	False	False
4	mean smoothness	1	True	False	False
3	mean area	1	True	False	False
2	mean perimeter	1	True	False	False
1	mean texture	1	True	False	False
13	area error	1	True	False	False
8	mean symmetry	2	False	True	True
11	texture error	3	False	True	False
18	symmetry error	3	False	True	False
14	smoothness error	4	False	True	False

```
[44]: !pip install tensorflow
```

Requirement already satisfied: tensorflow in c:\users\user\anaconda3\lib\site-packages (2.21.0)

Requirement already satisfied: absl-py>=1.0.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow) (2.5.0)

Requirement already satisfied: astunparse>=1.6.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow) (1.6.3)

Requirement already satisfied: flatbuffers>=25.9.23 in c:\users\user\anaconda3\lib\site-packages (from tensorflow) (25.12.19)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in c:\users\user\anaconda3\lib\site-packages (from tensorflow) (0.7.0)

Requirement already satisfied: google_pasta>=0.1.1 in

c:\users\user\anaconda3\lib\site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in c:\users\user\anaconda3\lib\site-
packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf<8.0.0,>=6.31.1 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (7.35.1)
Requirement already satisfied: requests<3,>=2.21.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (2.34.2)
Requirement already satisfied: setuptools in c:\users\user\anaconda3\lib\site-
packages (from tensorflow) (80.9.0)
Requirement already satisfied: six>=1.12.0 in c:\users\user\anaconda3\lib\site-
packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (1.17.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (1.84.0)
Requirement already satisfied: keras>=3.12.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (3.15.1)
Requirement already satisfied: numpy>=1.26.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (2.3.5)
Requirement already satisfied: h5py<3.15.0,>=3.11.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (3.14.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (0.6.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow)
(3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\user\anaconda3\lib\site-
packages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.26 in
c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow)
(2.5.0)
Requirement already satisfied: certifi>=2023.5.7 in
c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow)
(2026.5.20)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
c:\users\user\anaconda3\lib\site-packages (from astunparse>=1.6.0->tensorflow)
(0.45.1)
Requirement already satisfied: rich in c:\users\user\anaconda3\lib\site-packages
(from keras>=3.12.0->tensorflow) (14.2.0)
Requirement already satisfied: namex in c:\users\user\anaconda3\lib\site-

```

packages (from keras>=3.12.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in c:\users\user\anaconda3\lib\site-
packages (from keras>=3.12.0->tensorflow) (0.20.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in
c:\users\user\anaconda3\lib\site-packages (from rich->keras>=3.12.0->tensorflow)
(2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
c:\users\user\anaconda3\lib\site-packages (from rich->keras>=3.12.0->tensorflow)
(2.19.2)
Requirement already satisfied: mdurl~=0.1 in c:\users\user\anaconda3\lib\site-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.12.0->tensorflow) (0.1.2)

```

```

[45]: import tensorflow as tf
      from tensorflow import keras

```

```

[46]: models = keras.Sequential([
      keras.layers.Dense(80,activation= "relu",input_shape=(30,)),
      keras.layers.Dense(40,activation = "relu"),
      keras.layers.Dense(1, activation = "sigmoid")
    ])

```

```

C:\Users\User\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:107:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

[47]: # compile the model
      models.compile(optimizer = "adam",
                    loss = "binary_crossentropy",
                    metrics = ["accuracy"])
      #lets fit into the model
      history= models.fit(X_trained_scaled,
                        y_train,
                        validation_split = 0.2,
                        verbose = 1,
                        epochs = 100,
                        batch_size = 2)

```

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.
Please use WSL2 or the TensorFlow-DirectML plugin.

```

```

Epoch 1/100
182/182          4s 6ms/step -
accuracy: 0.9451 - loss: 0.1778 - val_accuracy: 0.9670 - val_loss: 0.0960
Epoch 2/100
182/182          1s 4ms/step -
accuracy: 0.9835 - loss: 0.0655 - val_accuracy: 0.9451 - val_loss: 0.0948

```

Epoch 3/100
182/182 1s 6ms/step -
accuracy: 0.9890 - loss: 0.0441 - val_accuracy: 0.9780 - val_loss: 0.0779

Epoch 4/100
182/182 1s 6ms/step -
accuracy: 0.9918 - loss: 0.0321 - val_accuracy: 0.9780 - val_loss: 0.0854

Epoch 5/100
182/182 1s 6ms/step -
accuracy: 0.9890 - loss: 0.0277 - val_accuracy: 0.9670 - val_loss: 0.0974

Epoch 6/100
182/182 1s 6ms/step -
accuracy: 0.9945 - loss: 0.0213 - val_accuracy: 0.9670 - val_loss: 0.0942

Epoch 7/100
182/182 1s 6ms/step -
accuracy: 0.9973 - loss: 0.0135 - val_accuracy: 0.9780 - val_loss: 0.0876

Epoch 8/100
182/182 1s 6ms/step -
accuracy: 0.9945 - loss: 0.0141 - val_accuracy: 0.9780 - val_loss: 0.0843

Epoch 9/100
182/182 1s 6ms/step -
accuracy: 0.9973 - loss: 0.0111 - val_accuracy: 0.9780 - val_loss: 0.1130

Epoch 10/100
182/182 1s 5ms/step -
accuracy: 0.9945 - loss: 0.0155 - val_accuracy: 0.9780 - val_loss: 0.0909

Epoch 11/100
182/182 1s 5ms/step -
accuracy: 0.9945 - loss: 0.0098 - val_accuracy: 0.9560 - val_loss: 0.1211

Epoch 12/100
182/182 1s 5ms/step -
accuracy: 0.9945 - loss: 0.0081 - val_accuracy: 0.9780 - val_loss: 0.0781

Epoch 13/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 0.0030 - val_accuracy: 0.9780 - val_loss: 0.0821

Epoch 14/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 0.0032 - val_accuracy: 0.9780 - val_loss: 0.0727

Epoch 15/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 0.0024 - val_accuracy: 0.9780 - val_loss: 0.0870

Epoch 16/100
182/182 1s 5ms/step -
accuracy: 0.9973 - loss: 0.0085 - val_accuracy: 0.9670 - val_loss: 0.0975

Epoch 17/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 0.0033 - val_accuracy: 0.9560 - val_loss: 0.1297

Epoch 18/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 0.0013 - val_accuracy: 0.9780 - val_loss: 0.0917

Epoch 19/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 8.0040e-04 - val_accuracy: 0.9560 - val_loss: 0.1078
Epoch 20/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 6.4725e-04 - val_accuracy: 0.9670 - val_loss: 0.1151
Epoch 21/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 5.4737e-04 - val_accuracy: 0.9670 - val_loss: 0.1105
Epoch 22/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 4.4042e-04 - val_accuracy: 0.9560 - val_loss: 0.1150
Epoch 23/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 4.1210e-04 - val_accuracy: 0.9560 - val_loss: 0.1231
Epoch 24/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 3.4131e-04 - val_accuracy: 0.9560 - val_loss: 0.1195
Epoch 25/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 2.7674e-04 - val_accuracy: 0.9560 - val_loss: 0.1256
Epoch 26/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 2.5675e-04 - val_accuracy: 0.9560 - val_loss: 0.1388
Epoch 27/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 2.2848e-04 - val_accuracy: 0.9560 - val_loss: 0.1325
Epoch 28/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.9145e-04 - val_accuracy: 0.9560 - val_loss: 0.1286
Epoch 29/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.7713e-04 - val_accuracy: 0.9560 - val_loss: 0.1399
Epoch 30/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.5081e-04 - val_accuracy: 0.9560 - val_loss: 0.1452
Epoch 31/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.3890e-04 - val_accuracy: 0.9560 - val_loss: 0.1418
Epoch 32/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.1382e-04 - val_accuracy: 0.9560 - val_loss: 0.1422
Epoch 33/100
182/182 1s 4ms/step -
accuracy: 1.0000 - loss: 9.6984e-05 - val_accuracy: 0.9560 - val_loss: 0.1505
Epoch 34/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 9.8529e-05 - val_accuracy: 0.9560 - val_loss: 0.1573

Epoch 35/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 8.1191e-05 - val_accuracy: 0.9560 - val_loss: 0.1538
Epoch 36/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 7.2732e-05 - val_accuracy: 0.9560 - val_loss: 0.1586
Epoch 37/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 6.5358e-05 - val_accuracy: 0.9560 - val_loss: 0.1576
Epoch 38/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 5.6108e-05 - val_accuracy: 0.9560 - val_loss: 0.1643
Epoch 39/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 4.7667e-05 - val_accuracy: 0.9560 - val_loss: 0.1635
Epoch 40/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 4.7111e-05 - val_accuracy: 0.9560 - val_loss: 0.1725
Epoch 41/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 3.9978e-05 - val_accuracy: 0.9560 - val_loss: 0.1655
Epoch 42/100
182/182 1s 7ms/step -
accuracy: 1.0000 - loss: 3.4657e-05 - val_accuracy: 0.9560 - val_loss: 0.1691
Epoch 43/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 3.2714e-05 - val_accuracy: 0.9560 - val_loss: 0.1669
Epoch 44/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 2.6675e-05 - val_accuracy: 0.9560 - val_loss: 0.1729
Epoch 45/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 2.7372e-05 - val_accuracy: 0.9560 - val_loss: 0.1815
Epoch 46/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 2.2949e-05 - val_accuracy: 0.9560 - val_loss: 0.1767
Epoch 47/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 2.0275e-05 - val_accuracy: 0.9560 - val_loss: 0.1821
Epoch 48/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 2.0334e-05 - val_accuracy: 0.9560 - val_loss: 0.1934
Epoch 49/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.6695e-05 - val_accuracy: 0.9560 - val_loss: 0.1911
Epoch 50/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.4732e-05 - val_accuracy: 0.9560 - val_loss: 0.1879

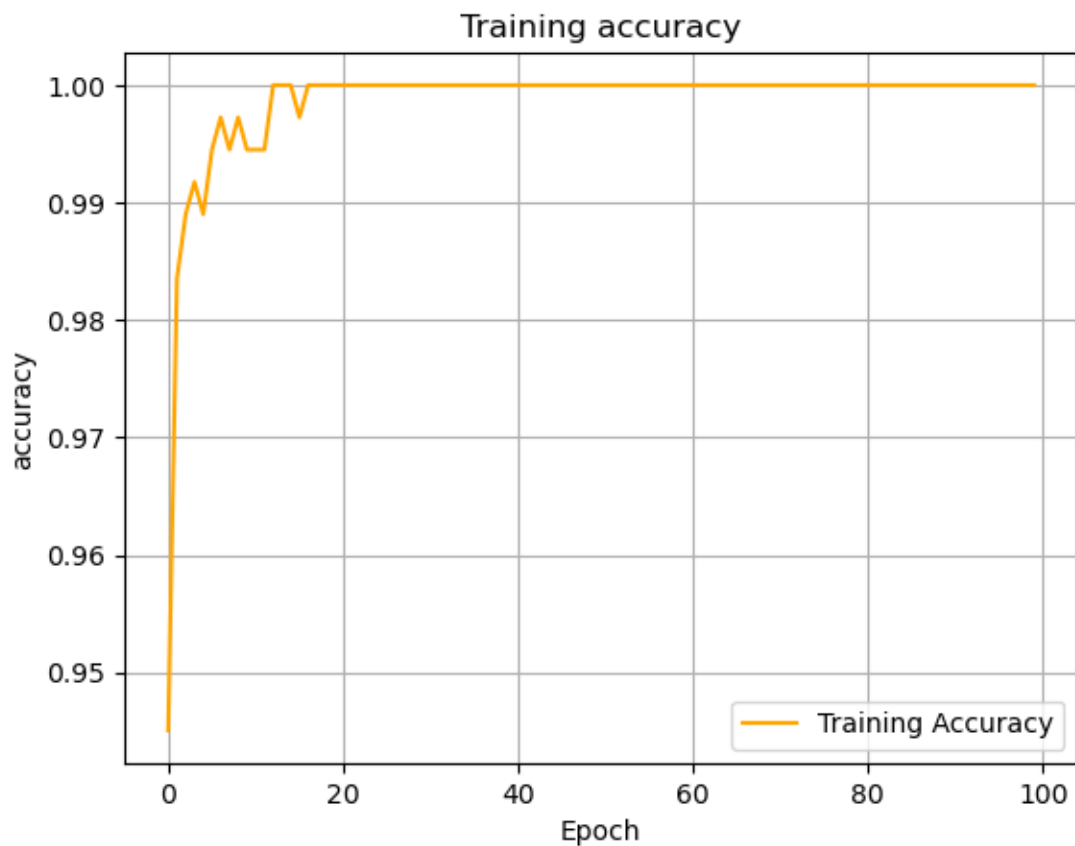
Epoch 51/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.3749e-05 - val_accuracy: 0.9560 - val_loss: 0.1913
Epoch 52/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.1587e-05 - val_accuracy: 0.9560 - val_loss: 0.1955
Epoch 53/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.0414e-05 - val_accuracy: 0.9560 - val_loss: 0.1989
Epoch 54/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 9.1189e-06 - val_accuracy: 0.9560 - val_loss: 0.2032
Epoch 55/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 8.4961e-06 - val_accuracy: 0.9560 - val_loss: 0.2048
Epoch 56/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 7.6972e-06 - val_accuracy: 0.9560 - val_loss: 0.2039
Epoch 57/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 6.9036e-06 - val_accuracy: 0.9560 - val_loss: 0.2054
Epoch 58/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 6.2677e-06 - val_accuracy: 0.9560 - val_loss: 0.2169
Epoch 59/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 5.3464e-06 - val_accuracy: 0.9560 - val_loss: 0.2139
Epoch 60/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 4.8712e-06 - val_accuracy: 0.9560 - val_loss: 0.2156
Epoch 61/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 4.2301e-06 - val_accuracy: 0.9560 - val_loss: 0.2198
Epoch 62/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 4.1466e-06 - val_accuracy: 0.9560 - val_loss: 0.2198
Epoch 63/100
182/182 1s 7ms/step -
accuracy: 1.0000 - loss: 4.0642e-06 - val_accuracy: 0.9560 - val_loss: 0.2216
Epoch 64/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 3.0248e-06 - val_accuracy: 0.9560 - val_loss: 0.2223
Epoch 65/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 3.0666e-06 - val_accuracy: 0.9560 - val_loss: 0.2300
Epoch 66/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 2.7903e-06 - val_accuracy: 0.9560 - val_loss: 0.2272

Epoch 67/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 2.2659e-06 - val_accuracy: 0.9560 - val_loss: 0.2279
Epoch 68/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.9811e-06 - val_accuracy: 0.9560 - val_loss: 0.2286
Epoch 69/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.9302e-06 - val_accuracy: 0.9560 - val_loss: 0.2329
Epoch 70/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.7401e-06 - val_accuracy: 0.9560 - val_loss: 0.2399
Epoch 71/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.4780e-06 - val_accuracy: 0.9560 - val_loss: 0.2406
Epoch 72/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.2853e-06 - val_accuracy: 0.9560 - val_loss: 0.2373
Epoch 73/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.3460e-06 - val_accuracy: 0.9560 - val_loss: 0.2520
Epoch 74/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.1135e-06 - val_accuracy: 0.9560 - val_loss: 0.2470
Epoch 75/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.0388e-06 - val_accuracy: 0.9560 - val_loss: 0.2424
Epoch 76/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 9.3623e-07 - val_accuracy: 0.9560 - val_loss: 0.2581
Epoch 77/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 7.7018e-07 - val_accuracy: 0.9560 - val_loss: 0.2529
Epoch 78/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 7.7790e-07 - val_accuracy: 0.9560 - val_loss: 0.2544
Epoch 79/100
182/182 1s 4ms/step -
accuracy: 1.0000 - loss: 6.3094e-07 - val_accuracy: 0.9560 - val_loss: 0.2566
Epoch 80/100
182/182 1s 4ms/step -
accuracy: 1.0000 - loss: 5.7424e-07 - val_accuracy: 0.9560 - val_loss: 0.2659
Epoch 81/100
182/182 1s 4ms/step -
accuracy: 1.0000 - loss: 5.5226e-07 - val_accuracy: 0.9560 - val_loss: 0.2589
Epoch 82/100
182/182 1s 4ms/step -
accuracy: 1.0000 - loss: 4.6707e-07 - val_accuracy: 0.9560 - val_loss: 0.2588

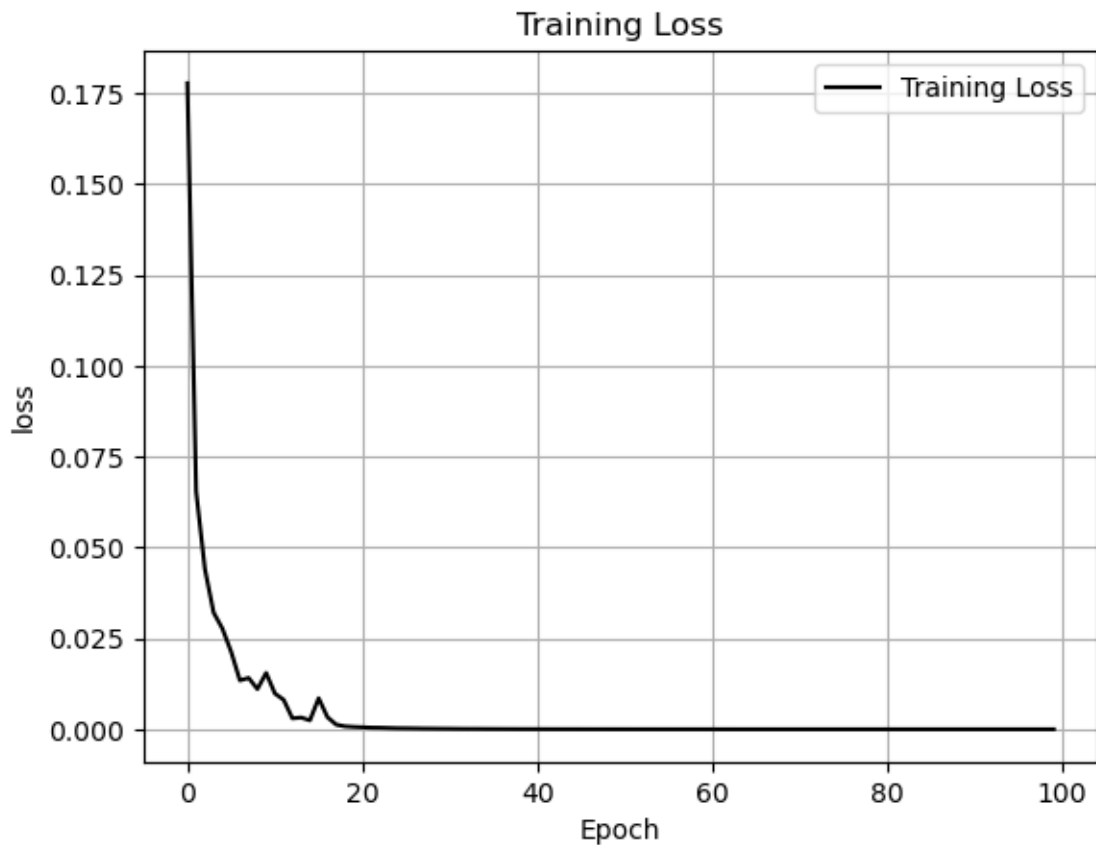
Epoch 83/100
182/182 1s 4ms/step -
accuracy: 1.0000 - loss: 4.3892e-07 - val_accuracy: 0.9560 - val_loss: 0.2655
Epoch 84/100
182/182 1s 3ms/step -
accuracy: 1.0000 - loss: 3.7844e-07 - val_accuracy: 0.9560 - val_loss: 0.2663
Epoch 85/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 3.4256e-07 - val_accuracy: 0.9560 - val_loss: 0.2670
Epoch 86/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 3.3592e-07 - val_accuracy: 0.9560 - val_loss: 0.2714
Epoch 87/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 2.9306e-07 - val_accuracy: 0.9560 - val_loss: 0.2888
Epoch 88/100
182/182 1s 4ms/step -
accuracy: 1.0000 - loss: 2.6561e-07 - val_accuracy: 0.9560 - val_loss: 0.2811
Epoch 89/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 2.1945e-07 - val_accuracy: 0.9560 - val_loss: 0.2819
Epoch 90/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 2.1869e-07 - val_accuracy: 0.9560 - val_loss: 0.2885
Epoch 91/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.9258e-07 - val_accuracy: 0.9560 - val_loss: 0.2816
Epoch 92/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.6102e-07 - val_accuracy: 0.9560 - val_loss: 0.2857
Epoch 93/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.5572e-07 - val_accuracy: 0.9560 - val_loss: 0.2898
Epoch 94/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.3720e-07 - val_accuracy: 0.9560 - val_loss: 0.2863
Epoch 95/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.2614e-07 - val_accuracy: 0.9560 - val_loss: 0.2891
Epoch 96/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.0434e-07 - val_accuracy: 0.9560 - val_loss: 0.2903
Epoch 97/100
182/182 1s 6ms/step -
accuracy: 1.0000 - loss: 1.0739e-07 - val_accuracy: 0.9560 - val_loss: 0.2909
Epoch 98/100
182/182 1s 5ms/step -
accuracy: 1.0000 - loss: 1.0239e-07 - val_accuracy: 0.9560 - val_loss: 0.3082


```
0.21557700634002686, 0.2197926640510559, 0.21981750428676605,
0.22159825265407562, 0.2222878336906433, 0.22999392449855804,
0.22721026837825775, 0.227914959192276, 0.22864611446857452, 0.2328762710094452,
0.23993971943855286, 0.24060305953025818, 0.23728299140930176,
0.25201666355133057, 0.2469765990972519, 0.24236823618412018,
0.25805696845054626, 0.25285327434539795, 0.2543739974498749,
0.25662970542907715, 0.2658596336841583, 0.25894105434417725, 0.25881227850914,
0.26547133922576904, 0.2663392424583435, 0.26701778173446655,
0.2714475989341736, 0.28880828619003296, 0.2810913622379303, 0.2819308042526245,
0.28850021958351135, 0.2816406786441803, 0.2857224941253662, 0.2898308336734772,
0.2862764298915863, 0.2891412377357483, 0.2902599275112152, 0.2908862233161926,
0.30820825695991516, 0.30423253774642944, 0.30757564306259155]}
```

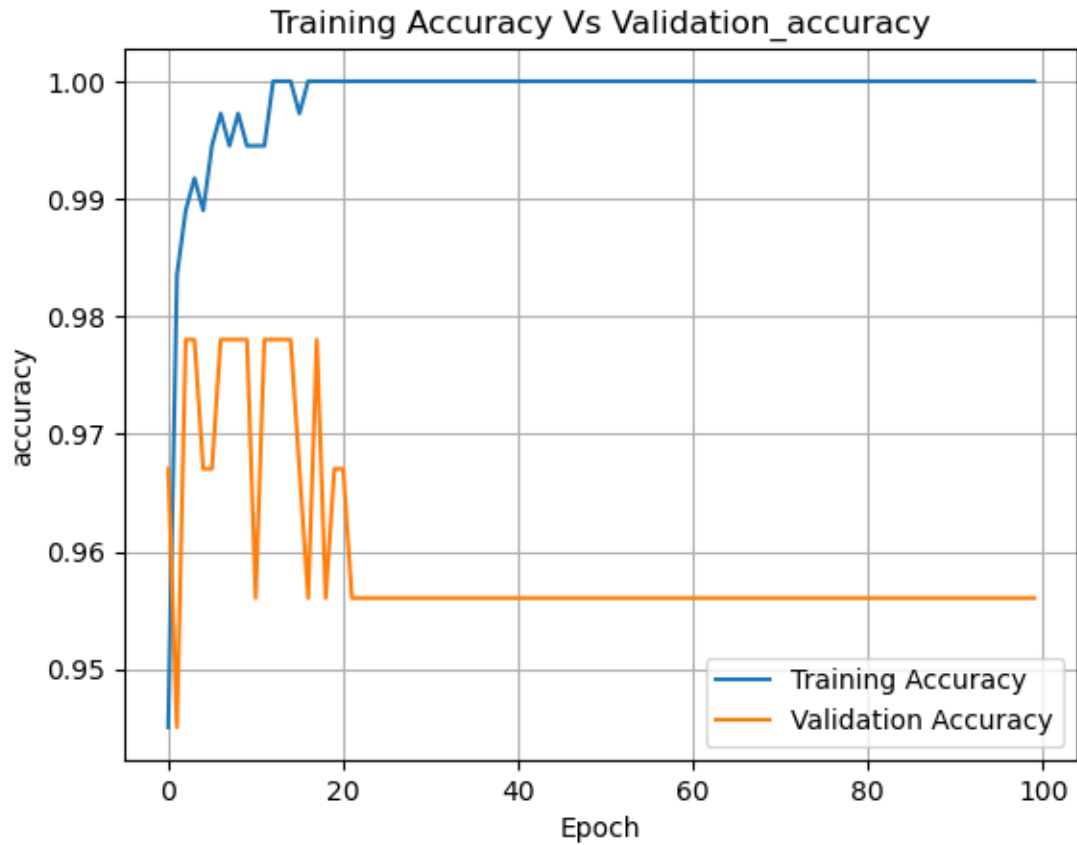
```
[49]: plt.plot(history.history["accuracy"], label = "Training Accuracy",color = "orange")
plt.xlabel("Epoch")
plt.ylabel("accuracy")
plt.title("Training accuracy")
plt.legend()
plt.grid(True)
plt.show()
```



```
[50]: plt.plot(history.history["loss"], label = "Training Loss", color = "black")
plt.xlabel("Epoch")
plt.ylabel("loss")
plt.title("Training Loss")
plt.legend()
plt.grid(True)
plt.show()
```



```
[51]: plt.plot(history.history["accuracy"], label = "Training Accuracy")
plt.plot(history.history["val_accuracy"], label = "Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("accuracy")
plt.title("Training Accuracy Vs Validation_accuracy")
plt.legend()
plt.grid(True)
plt.show()
```



```
[52]: plt.plot(history.history["loss"],label = "Training Loss")
plt.plot(history.history["val_loss"],label = "Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("loss")
plt.title("Training Loss Vs Validation_loss")
plt.legend()
plt.grid(True)
plt.show()
```



```
[53]: print(history.history.keys())
```

```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

```
[54]: loss, accuracy = models.evaluate(X_trained_scaled, y_train)
print(f"Loss : {loss}")
print(f"accuracy :{accuracy}")
```

```
15/15          0s 8ms/step -
accuracy: 0.9912 - loss: 0.0615
Loss : 0.06151517480611801
accuracy :0.9912087917327881
```